Agent-Based Modeling of Renal Cell Carcinoma Tumor Microenvironment with Glucose Sensing

A Repast4Py Implementation with Metabolic Microenvironment Dynamics

MASL Project

March 13, 2026

Abstract

This report presents a comprehensive agent-based model (ABM) of the Renal Cell Carcinoma (RCC) tumor microenvironment, implemented using the Repast4Py framework for distributed parallel simulation. The model captures interactions among 18 distinct agent types—including tumor cells, various immune cell populations, blood vessels, sex hormones, and cytokines—within a 3D spatial grid. A novel glucose concentration field is overlaid on the discrete agent grid, modeling metabolic dynamics via Fick's diffusion, the Warburg effect for tumor metabolism, and gradient-based immune cell chemotaxis. The implementation ports an existing Mesa-based model to Repast4Py, leveraging MPI-based parallelism, and extends it with 8 additional glucose-related parameters. The model incorporates a 16-gene DNA mutation system, PD-1/PD-L1 immune checkpoint interactions, ICI and TKI treatment modalities, sex hormone effects on immune response, and Bayesian parameter optimization via Optuna against ARON-1 clinical trial data. This document describes the biological motivation, mathematical formulation, software architecture, and implementation details of the complete system.

Contents

1 Introduction **6**
1.1 Renal Cell Carcinoma: Epidemiology and Clinical Significance 6
1.2 The Tumor Microenvironment 6
1.3 Sex Differences in RCC 7
1.4 Motivation for Computational Modeling 7
1.5 Agent-Based Modeling Approach 8
1.6 Project Objectives 9
1.7 Report Structure 9

2 Theoretical Background **10**
2.1 Agent-Based Modeling 10
2.1.1 Formalism and Core Concepts 10
2.1.2 Advantages for Biological Systems 11
2.1.3 The Repast4Py Framework 11
2.1.4 Comparison with Mesa 12
2.2 Immunology of Renal Cell Carcinoma 12
2.2.1 Innate and Adaptive Immunity 12
2.2.2 T Cell Subtypes 13
2.2.3 Antigen-Presenting Cells 14
2.2.4 Macrophages 14
2.2.5 Natural Killer Cells 14
2.2.6 Mast Cells and Neutrophils 15
2.2.7 Immune Checkpoints: The PD-1/PD-L1 Pathway 15
2.2.8 Tumor Immune Evasion Mechanisms 15
2.3 Tumor Metabolism 16
2.3.1 The Warburg Effect 16
2.3.2 Metabolic Competition in the TME 16
2.3.3 Glucose as a Limiting Factor for Immune Function 16
2.4 Treatment Modalities 17
2.4.1 Immune Checkpoint Inhibitors (ICI) 17
2.4.2 Tyrosine Kinase Inhibitors (TKI) 17
2.4.3 Combination ICI+TKI Therapy 18

3 Model Description **19**
3.1 Overview 19
3.1.1 Spatial Environment 19
3.1.2 Agent Types 19
3.1.3 Time Step Structure 20

3.2 Agent Hierarchy . . . 21
3.2.1 Base Cell Class . . . 21
3.2.2 TCell Base Class . . . 22
3.2.3 Concrete Agent Classes . . . 22
3.3 Effect System . . . 23
3.3.1 Effect Fields . . . 23
3.3.2 BMI Normalization . . . 23
3.3.3 Effect Collection and Aggregation . . . 24
3.4 Agent Interactions . . . 24
3.5 Movement . . . 27
3.5.1 Deferred Movement Pattern . . . 27
3.5.2 Directed Movement: `move_towards()` . . . 28
3.5.3 Glucose Chemotaxis . . . 28
3.5.4 Sex Hormone Diffusion . . . 28
3.6 Terminal Conditions . . . 29

4 Glucose Microenvironment System **30**
4.1 Biological Motivation . . . 30
4.2 Mathematical Formulation . . . 31
4.2.1 Discrete 3D Laplacian . . . 31
4.2.2 Time Integration . . . 31
4.2.3 Stability Analysis . . . 32
4.2.4 Boundary Conditions . . . 32
4.3 Source Model . . . 32
4.4 Consumption Model . . . 32
4.4.1 Tumor Cell Consumption . . . 32
4.4.2 Immune Cell Consumption . . . 33
4.4.3 Glucose-Dependent Growth Factor . . . 33
4.4.4 Glucose-Dependent Immune Function . . . 33
4.5 Gradient and Chemotaxis . . . 33
4.5.1 Gradient Computation . . . 33
4.5.2 Discretized Gradient Step . . . 34
4.5.3 Chemotaxis Behavior . . . 34
4.6 Implementation . . . 34
4.6.1 Data Structure . . . 34
4.6.2 Convolution Kernel . . . 34
4.6.3 Step Function . . . 35
4.6.4 Statistics Methods . . . 35

5 DNA and Mutation System **36**
5.1 Gene Model . . . 36
5.1.1 Gene Catalogue . . . 36
5.1.2 DNA Sequence Structure . . . 37
5.2 Codon Translation . . . 37
5.2.1 Codon Table . . . 38
5.2.2 Translation: `gene_to_protein()` . . . 38
5.2.3 Reverse Translation: `protein_to_gene()` . . . 38
5.3 Mutation Mechanics . . . 38
5.3.1 Genomic Instability . . . 38

5.3.2 DNA Duplication with Point Mutations . . . 39
5.3.3 Injected Mutations . . . 39
5.4 Gene Expression and Phenotypic Effects . . . 39
5.4.1 Promoter Expression . . . 39
5.4.2 Mutation Score . . . 39
5.4.3 Gene Mutation–Expression Product . . . 40
5.4.4 Five Derived Phenotypic Parameters . . . 40
5.5 Neoantigen Generation . . . 41
5.5.1 Neoantigen Definition . . . 41
5.5.2 Self-Antigens . . . 41
5.5.3 TCR Matching . . . 41

6 Treatment System **43**
6.1 Drug Architecture . . . 43
6.1.1 Abstract Drug Base Class . . . 43
6.1.2 Treatment Composition . . . 43
6.1.3 Treatment Activation . . . 44
6.2 Immune Checkpoint Inhibitors (ICI) . . . 44
6.2.1 Biological Basis . . . 44
6.2.2 Implementation . . . 44
6.3 Tyrosine Kinase Inhibitors (TKI) . . . 45
6.3.1 Biological Basis . . . 45
6.3.2 Implementation . . . 45
6.4 Combination Therapy . . . 46
6.4.1 ICI + TKI . . . 46
6.4.2 Synergistic Mechanisms . . . 46
6.4.3 Clinical Evidence . . . 47

7 Sex Hormone System **48**
7.1 Biological Basis . . . 48
7.1.1 Sex Differences in Renal Cell Carcinoma . . . 48
7.1.2 Estrogen . . . 48
7.1.3 Testosterone . . . 48
7.1.4 Progesterone . . . 49
7.2 Hormone Agent Model . . . 49
7.2.1 Agent Type and Inheritance . . . 49
7.2.2 Hormone Types . . . 49
7.2.3 Spawning Mechanism . . . 49
7.2.4 Hormone Diffusion . . . 50
7.3 Hormone Effects on T Cells . . . 50
7.3.1 Hormone Perception . . . 50
7.3.2 Stimulation Level . . . 50
7.3.3 Hormonal Influence Effects . . . 50
7.3.4 Concentration-Dependent Effect Mapping . . . 51
7.4 Sex Drift . . . 52
7.4.1 Biological Basis . . . 52
7.4.2 Implementation . . . 52
7.4.3 Interaction with Hormonal Effects . . . 53

8 Implementation **54**
8.1 Framework Selection . . . 54
8.1.1 Rationale for Repast4Py . . . 54
8.1.2 Comparison with Mesa . . . 55
8.2 Architecture . . . 55
8.2.1 Project Structure . . . 55
8.2.2 Plain Python Model Class . . . 56
8.3 Spatial Index Design . . . 57
8.3.1 Problem . . . 57
8.3.2 Solution . . . 57
8.3.3 Grid Utilities Module . . . 57
8.4 Agent Serialization . . . 58
8.4.1 Agent Identity in Repast4Py . . . 58
8.4.2 Agent Type Registry . . . 58
8.4.3 The `save()` Method . . . 59
8.4.4 Type Identification . . . 59
8.5 Mesa to Repast4Py Porting . . . 60
8.5.1 Random Number Generation . . . 60
8.5.2 Class-Level State to Model-Level State . . . 60
8.5.3 Deferred Movement . . . 61
8.6 Glucose Field Implementation . . . 61
8.6.1 Data Structure . . . 61
8.6.2 Diffusion Kernel . . . 61
8.6.3 Boundary Conditions . . . 62
8.6.4 Separation from Repast4Py Value Layers . . . 62
8.7 Entry Point and Execution . . . 62
8.7.1 Execution Command . . . 63
8.7.2 Simulation Loop . . . 63

9 Parameters and Optimization **65**
9.1 Parameter Categories . . . 65
9.1.1 ModelParameters . . . 65
9.1.2 PatientParameters . . . 65
9.1.3 WeightParameters . . . 66
9.2 Weight Parameters . . . 66
9.2.1 BMI Weights (4 parameters) . . . 67
9.2.2 Macrophage M1 Weights (6 weights + 2 biases) . . . 67
9.2.3 Macrophage M2 Weights (5 weights + 1 bias) . . . 68
9.2.4 Tumor Cell Weights (8 weights + 3 biases) . . . 68
9.2.5 CD4+ T Cell Weights (9 weights + 3 biases) . . . 69
9.2.6 Cytotoxic T Cell Weights (5 weights + 2 biases) . . . 69
9.2.7 Mast Cell Weights (6 parameters) . . . 70
9.2.8 NK Cell Weights (3 weights + 1 bias) . . . 70
9.2.9 Plasmacytoid DC Weights (6 weights + 1 bias) . . . 70
9.2.10 Treg Weights (5 parameters) . . . 71
9.2.11 Global Weights (4+ parameters) . . . 71
9.3 Glucose Parameters . . . 71
9.3.1 Parameter Interactions . . . 73

9.4 Bayesian Optimization . . . 73
9.4.1 Framework . . . 73
9.4.2 Objective Function . . . 73
9.4.3 Search Space . . . 74
9.4.4 Optimization Configuration . . . 74
9.5 Clinical Dataset . . . 74
9.5.1 ARON-1 Study . . . 74
9.5.2 Dataset Structure . . . 75

10 Simulation Results **76**
10.1 Baseline Simulation . . . 76
10.1.1 Expected Tumor Dynamics . . . 76
10.1.2 Expected Immune Response . . . 76
10.1.3 Glucose Field Behavior . . . 77
10.2 Treatment Scenarios . . . 77
10.2.1 ICI Only . . . 77
10.2.2 TKI Only . . . 78
10.2.3 ICI+TKI Combination . . . 78
10.2.4 Expected Visualizations . . . 78
10.3 Sex-Specific Responses . . . 79
10.3.1 Male vs. Female Comparison . . . 79
10.3.2 Expected Outcomes . . . 79
10.4 Glucose Field Dynamics . . . 79
10.4.1 Initial State . . . 79
10.4.2 Source–Diffusion–Consumption Cycle . . . 79
10.4.3 Spatial Profile Evolution . . . 80
10.4.4 Warburg Effect Manifestation . . . 80
10.5 Validation . . . 81
10.5.1 Comparison Against ARON-1 Data . . . 81
10.5.2 Expected Accuracy . . . 81
10.5.3 Parameter Sensitivity Analysis . . . 81

11 Conclusion **83**
11.1 Summary of Contributions . . . 83
11.2 Model Limitations . . . 84
11.3 Future Work . . . 85

Chapter 1

Introduction

1.1 Renal Cell Carcinoma: Epidemiology and Clinical Significance

Renal cell carcinoma (RCC) is the third most common urological malignancy and accounts for approximately 90% of all kidney cancers [13]. Globally, RCC is responsible for roughly 400,000 new diagnoses per year, with incidence rates that have been steadily rising over the past several decades—a trend attributed in part to the increased use of abdominal imaging and in part to the growing prevalence of established risk factors such as obesity, hypertension, and tobacco use [7].

The prognosis for RCC is strongly stage-dependent. When the disease is detected in a localized state—confined to the kidney—five-year survival rates range from 50% to 75%, and for early-stage tumors treated with partial or radical nephrectomy, outcomes can be even more favorable. However, RCC has a marked propensity for late diagnosis: approximately one-third of patients present with locally advanced or metastatic disease at the time of initial detection. For metastatic RCC (mRCC), the five-year survival rate drops to below 15%, underscoring the aggressive nature of late-stage disease and the limitations of currently available systemic therapies [12].

Clear cell RCC (ccRCC) constitutes 70–80% of all RCC cases and is characterized by mutations in the *VHL* gene, which lead to constitutive activation of the hypoxia-inducible factor (HIF) pathway and consequent overexpression of vascular endothelial growth factor (VEGF). This molecular hallmark has made ccRCC a prototypical target for anti-angiogenic therapies, while its relatively high immunogenicity—evidenced by dense immune infiltrates and occasional spontaneous regressions—has positioned it as one of the tumors most responsive to immunotherapy [11].

1.2 The Tumor Microenvironment

The tumor microenvironment (TME) is a complex, spatially heterogeneous ecosystem that extends far beyond the neoplastic cells themselves. It encompasses:

- **Tumor cells** with diverse genetic and phenotypic profiles arising from genomic instability and clonal evolution.
- **Immune cells** of both innate and adaptive lineages, including cytotoxic T lymphocytes (CTLs), helper T cells (Th1 and Th2), regulatory T cells (Tregs), dendritic cells (DCs), macrophages (M1 and M2 polarization states), natural killer (NK) cells,

mast cells, and neutrophils.

- **Stromal cells** such as adipocytes and fibroblasts that provide structural and metabolic support.
- **Vasculature**, both pre-existing and tumor-induced (via angiogenesis), which supplies oxygen, glucose, and other nutrients while also serving as a route for immune cell trafficking and hormone delivery.
- **Signaling molecules** including cytokines, chemokines, and sex hormones that mediate cell-cell communication over varying spatial scales.

The interactions among these components are bidirectional and highly nonlinear. Immune cells can recognize and kill tumor cells, but tumors can also co-opt immune cells—for instance, by promoting M2 macrophage polarization or Treg expansion—to create an immunosuppressive niche. Understanding these dynamics requires approaches that can capture spatial heterogeneity, stochasticity, and the emergent behaviors that arise from local cell-cell interactions [7].

1.3 Sex Differences in RCC

A striking and incompletely understood feature of RCC is its pronounced sex bias: males exhibit approximately twice the incidence of females across virtually all populations studied. While differential exposure to risk factors (e.g., smoking, occupational hazards) accounts for part of this disparity, accumulating evidence implicates sex hormones—particularly estrogen, testosterone, and progesterone—as modulators of both tumor biology and immune response.

Estrogen has been shown to enhance several arms of the anti-tumor immune response. At low concentrations, estrogen promotes Th1 polarization, increases interferon-gamma (IFN-γ) production, upregulates MHC class I and II expression, and enhances cytotoxic T cell activity. At higher concentrations, however, estrogen can shift the balance toward Th2 responses, increase regulatory T cell differentiation, and upregulate immune checkpoint molecules such as CTLA-4 and PD-1, thereby dampening anti-tumor immunity. Testosterone, conversely, tends to suppress immune responses at elevated levels, reducing Th1 proliferation and IFN-γ production while promoting checkpoint-mediated immune evasion [3].

These observations have motivated the ARON-1 clinical study, which investigates whether sex-specific immune landscapes translate into differential responses to combination immunotherapy-targeted therapy regimens in RCC patients. The ARON-1 study provides the clinical context and validation data for the computational model presented in this work, and its findings regarding sex-dependent treatment outcomes serve as key calibration targets for the simulation [13].

1.4 Motivation for Computational Modeling

The complexity of the RCC tumor microenvironment—with its heterogeneous cell populations, spatial organization, stochastic interactions, metabolic competition, hormonal modulation, and treatment effects—presents a formidable challenge for traditional analytical and experimental approaches. Clinical trials, while indispensable, are expensive, time-consuming, and inherently limited in the number of treatment combinations, dosing schedules, and patient stratifications they can explore. *In vitro* and *in vivo* models

capture some aspects of tumor-immune dynamics but cannot easily reproduce the full complexity of the human TME.

Computational modeling offers a complementary approach that addresses several of these limitations:

1. **In silico experimentation.** Simulations allow researchers to conduct virtual experiments that would be impractical, unethical, or prohibitively expensive in clinical settings. Thousands of virtual patients with varying immune profiles, hormonal states, and treatment regimens can be simulated in hours rather than years.
2. **Parameter exploration.** The model's behavior can be systematically mapped across high-dimensional parameter spaces, identifying which biological variables most strongly influence treatment outcomes.
3. **Treatment optimization.** By coupling the simulation with optimization algorithms, it becomes possible to search for treatment schedules and drug combinations that maximize predicted patient survival for specific patient subgroups.
4. **Hypothesis generation.** Emergent behaviors observed in simulation—such as unexpected immune cell population dynamics or treatment resistance mechanisms—can generate testable hypotheses for subsequent experimental validation.

Previous work by Merelli et al. [9] and the MASL research group [6] demonstrated the feasibility of agent-based approaches for modeling the RCC microenvironment, establishing a Mesa-based prototype that captured the core immune-tumor interactions. The present work extends this foundation substantially.

1.5 Agent-Based Modeling Approach

Agent-based modeling (ABM) is a computational paradigm in which a system is represented as a collection of autonomous entities—agents—that interact with each other and their environment according to defined rules. The macroscopic behavior of the system emerges from the aggregate of these local interactions, rather than being prescribed by top-down equations.

ABM is particularly well suited to modeling the tumor microenvironment for several reasons:

- **Natural representation of heterogeneity.** Each agent (cell) can carry its own state: a unique genome with specific mutations, a particular activation level, a spatial position, and accumulated effects from neighbors. This heterogeneity is intrinsic to the formalism rather than an approximation.
- **Spatial dynamics.** Agents occupy positions in a 3D grid and interact with their local neighborhood, capturing the spatial structure of the TME—including the formation of immune-excluded regions, angiogenesis-driven vascular networks, and glucose gradients.
- **Stochasticity.** Biological processes such as cell division, apoptosis, immune recognition, and mutation are inherently probabilistic. ABM naturally incorporates stochastic decision-making at the individual agent level.
- **Emergent behavior.** Complex phenomena such as immune evasion, tumor dormancy, and treatment resistance can emerge from simple local rules without being explicitly programmed, providing mechanistic insight into system-level dynamics.

The choice of Repast4Py [2] as the simulation framework is motivated by its support for MPI-based parallel execution, which enables the model to scale to larger grid sizes and agent populations than would be feasible with single-process frameworks such as Mesa

[8]. Repast4Py provides core abstractions—`SharedContext`, `SharedGrid`, and an MPI-aware scheduling mechanism—that allow the model to distribute agents across multiple processes while maintaining consistent spatial queries and inter-agent communication.

1.6 Project Objectives

The work presented in this report pursues four principal objectives:

1. **Port the existing Mesa model to Repast4Py for MPI parallelism.** The original RCC agent-based model was implemented using the Mesa framework, which is restricted to single-process execution. This project re-implements the model using Repast4Py, restructuring the agent hierarchy, spatial indexing, and scheduling to leverage MPI-based distribution. The port preserves the 18 agent types and their interaction logic while adapting to Repast4Py's `Agent`, `SharedContext`, and `SharedGrid` abstractions.

2. **Add a glucose microenvironment sensing layer.** A novel continuous 3D glucose concentration field is overlaid on the discrete agent grid. Blood vessels inject glucose, which diffuses according to Fick's law and decays over time. Tumor cells consume glucose at elevated rates consistent with the Warburg effect, creating local depletion zones. Immune cells both consume glucose and use its gradient for chemotaxis when no direct target is available. This metabolic layer introduces 8 new parameters governing glucose dynamics and their coupling to agent behavior.

3. **Validate against ARON-1 clinical data.** The model's predictions regarding tumor growth trajectories, immune cell population dynamics, and treatment outcomes are compared against data from the ARON-1 clinical trial, with particular attention to sex-stratified responses to ICI, TKI, and combination ICI+TKI therapy.

4. **Optimize parameters via Bayesian optimization.** An Optuna-based [1] Bayesian optimization pipeline is implemented to search the model's parameter space for configurations that best reproduce clinical observations. This optimization operates over the model's weight parameters, including the newly introduced glucose-related parameters, and targets fitness metrics derived from ARON-1 patient outcomes.

1.7 Report Structure

The remainder of this report is organized as follows. Chapter 2 provides the theoretical background on agent-based modeling, RCC immunology, tumor metabolism, and treatment modalities. Chapter 3 presents a detailed description of the model architecture, including the agent hierarchy, effect system, interaction rules, and terminal conditions. Chapter 4 describes the glucose concentration field and its integration with agent behavior. Chapter 5 details the DNA mutation system that governs tumor evolution and antigen presentation. Chapter 6 covers the treatment system, including ICI and TKI drug mechanisms. Chapter 7 describes the sex hormone system and its modulation of immune responses. Chapter 8 discusses implementation details and the software architecture. Chapter 9 catalogs the full parameter set. Chapter 10 presents simulation results and validation against clinical data. Chapter 11 concludes with a summary and directions for future work.

Chapter 2

Theoretical Background

This chapter presents the theoretical foundations upon which the RCC agent-based model is constructed. We begin with a formal introduction to agent-based modeling and the Repast4Py framework (Section 2.1), then review the immunology of renal cell carcinoma with emphasis on the cell types represented in the model (Section 2.2), discuss the metabolic characteristics of the tumor microenvironment (Section 2.3), and conclude with an overview of the treatment modalities implemented in the simulation (Section 2.4).

2.1 Agent-Based Modeling

2.1.1 Formalism and Core Concepts

An agent-based model (ABM) is a computational model consisting of three fundamental components:

1. **Agents.** Autonomous entities characterized by a set of internal state variables and behavioral rules. Each agent perceives its local environment, makes decisions based on its state and perception, and executes actions that may modify its own state, the state of other agents, or the environment.

2. **Environment.** A shared space—typically a discrete grid or continuous domain—in which agents are situated. The environment may carry its own state variables (e.g., chemical concentrations, nutrient fields) that evolve according to physical laws and are modified by agent actions.

3. **Rules.** The behavioral logic governing agent actions. Rules are typically local (each agent acts based on information in its neighborhood) and stochastic (outcomes are drawn from probability distributions parameterized by the agent's state and accumulated effects).

The distinguishing feature of ABM is that system-level behavior is *emergent*: it arises from the aggregate of individual agent actions and interactions rather than being prescribed by global equations. This makes ABM particularly powerful for systems characterized by heterogeneity, spatial structure, and nonlinear feedback—properties that are hallmarks of the tumor microenvironment.

Formally, at each discrete time step t, the state of agent i is updated according to:

$$s_i(t+1) = f\big(s_i(t),\, \mathcal{N}_i(t),\, E(t),\, \xi_i(t)\big) \tag{2.1}$$

where $s_i(t)$ is the agent's internal state, $\mathcal{N}_i(t)$ is the set of neighboring agents within a defined radius, $E(t)$ is the local environmental state (e.g., glucose concentration), and $\xi_i(t)$ represents stochastic noise. The environment itself evolves as:

$$E(t+1) = g\big(E(t),\, \{a_i(t)\}_{i=1}^{N}\big) \tag{2.2}$$

where $\{a_i(t)\}$ denotes the actions taken by all N agents at time t.

2.1.2 Advantages for Biological Systems

ABM offers several advantages for modeling biological systems in general and the tumor microenvironment in particular:

- **Heterogeneity.** Unlike equation-based models that track population-level averages, ABM represents each cell individually. In the present model, each tumor cell carries a unique 16-gene DNA sequence that determines its antigen presentation, proliferation potential, and susceptibility to immune attack. This cell-level heterogeneity is critical for capturing clonal evolution and immune escape.

- **Spatial dynamics.** Agent positions in a 3D grid determine which cells can interact. Spatial phenomena such as immune exclusion zones around tumors, angiogenesis-driven vascular networks, and nutrient gradients emerge naturally from local interactions.

- **Stochasticity.** Biological events—mitosis, apoptosis, antigen recognition, cytokine release—are inherently probabilistic. ABM incorporates stochasticity at the individual agent level, and ensemble statistics over multiple simulation runs capture the variability observed in clinical data.

- **Modularity.** New agent types, interaction rules, or environmental layers (such as the glucose field introduced in this work) can be added without restructuring the entire model, facilitating iterative model development.

2.1.3 The Repast4Py Framework

Repast4Py [2] is a Python-based agent-based modeling framework designed for large-scale distributed simulations. It extends the Repast HPC (High-Performance Computing) toolkit to the Python ecosystem, leveraging MPI (Message Passing Interface) [4] for inter-process communication. The key abstractions provided by Repast4Py are:

- `repast4py.core.Agent`. The base class for all agents. Each agent is identified by a tuple `(id, type, rank)` where `id` is a locally unique integer, `type` is an integer agent type identifier, and `rank` is the MPI rank of the process that owns the agent.

- `SharedContext`. A distributed container for agents. The context manages agent ownership across MPI ranks and provides methods for adding, removing, and iterating over agents.

- `SharedGrid`. A distributed spatial projection that maps agents to discrete grid coordinates. Repast4Py supports multiple grid configurations:

- `BorderType.Sticky`: agents that attempt to move beyond grid boundaries are clamped to the boundary (used in the present model).
- `OccupancyType.Multiple`: multiple agents can occupy the same grid cell (necessary for a dense microenvironment).

- **Schedule runner.** The scheduling mechanism, initialized via `schedule.init_schedule_runner`, manages the execution order of model steps across MPI ranks, ensuring synchronization where required.

2.1.4 Comparison with Mesa

The original RCC model was implemented in Mesa [8], a widely used Python ABM framework. Table 2.1 summarizes the key differences between Mesa and Repast4Py that motivated the port.

Table 2.1: Comparison of Mesa and Repast4Py for the RCC model.

Feature	**Mesa**	**Repast4Py**
Execution model	Single-process	MPI-distributed
Grid type	`MultiGrid` (2D/3D)	`SharedGrid` (distributed)
Agent base class	`mesa.Agent`	`repast4py.core.Agent`
Scheduling	`RandomActivation`	`schedule.init_schedule_runner`
Scalability	Limited by single core	Scales across MPI ranks
Neighbor queries	Built-in grid methods	Custom spatial index
Serialization	Not required	`save()` / restore functions

The port to Repast4Py required significant restructuring. In Mesa, neighbor queries are handled by the grid object directly (e.g., `grid.get_neighbors()`). In the Repast4Py implementation, a custom `spatial_index` dictionary maps grid positions to agent lists, and utility functions in `grid_utils` provide 3D neighborhood queries, empty-cell detection, and Moore neighborhood enumeration. Additionally, Repast4Py requires explicit `save()` methods on agents for MPI serialization, and the model manages agent lifecycle (creation, movement, removal) through centralized methods on the `RCCModel` class rather than through Mesa's scheduler.

2.2 Immunology of Renal Cell Carcinoma

2.2.1 Innate and Adaptive Immunity

The immune system's response to cancer involves both the innate and adaptive arms. Innate immunity provides a rapid, non-specific first line of defense through cells such as natural killer (NK) cells, macrophages, dendritic cells, mast cells, and neutrophils. Adaptive immunity, mediated primarily by T and B lymphocytes, provides antigen-specific responses that can be highly effective against tumors expressing recognizable neoantigens.

In the context of RCC, the interplay between innate and adaptive immunity is critical. Dendritic cells serve as the bridge between the two arms: they phagocytose tumor cells,

process their antigens, and present them to naive T cells, thereby initiating adaptive responses. The effectiveness of this chain depends on the tumor's ability to present antigens (governed by MHC expression and the antigen-processing machinery) and the availability and activation state of T cells in the tumor neighborhood.

2.2.2 T Cell Subtypes

The model distinguishes six T cell populations, reflecting the major functional categories relevant to anti-tumor immunity:

1. **$CD8^+$ Cytotoxic T Cells (CTLs).** The primary effectors of adaptive anti-tumor immunity. CTLs recognize tumor-specific antigens presented on MHC class I molecules and kill target cells through the release of perforin and granzymes. Their effectiveness is modulated by activation level, PD-1/PD-L1 checkpoint interactions, and local glucose availability.

2. **$CD8^+$ Naive T Cells.** Antigen-inexperienced $CD8^+$ cells residing in the microenvironment. Upon encountering a dendritic cell presenting a matching neoantigen (determined by TCR-antigen Hamming distance), they differentiate into cytotoxic T cells.

3. **$CD4^+$ Naive T Cells.** Precursors to helper and regulatory T cells. Depending on the local cytokine milieu and hormonal environment, they can differentiate into Th1, Th2, or Treg cells. Antigen presentation by dendritic cells or macrophages triggers differentiation into Th1 or Th2 with equal probability, conditional on TCR-antigen matching.

4. **$CD4^+$ Th1 Helper T Cells.** Pro-inflammatory cells that produce IFN-γ and promote anti-tumor immunity. In the model, Th1 cells recruit M1 macrophages and dendritic cells, promote tumor apoptosis, and can proliferate under the influence of accumulated Th1-proliferation effects. Their proliferation is modulated by sex hormones, with low estrogen favoring Th1 expansion and high estrogen suppressing it.

5. **$CD4^+$ Th2 Helper T Cells.** Cells associated with a humoral immune response that produce IL-4, IL-5, and IL-13. In the tumor context, Th2 cells recruit M1 macrophages and cytotoxic T cells, but they also exert a mild pro-tumorigenic effect by increasing tumor growth. Th2 proliferation is enhanced by estrogen and progesterone, consistent with the shift toward Th2-dominant responses at high hormonal levels.

6. **Regulatory T Cells (Tregs).** Immunosuppressive cells that play a central role in tumor immune evasion. Tregs inhibit cytotoxic T cell killing (reducing `t_kill_rate_effect`), suppress T cell proliferation and activation, increase T cell apoptosis, and reduce dendritic cell phagocytosis. They differentiate from $CD4^+$ naive cells under the influence of the `treg_differentiation_effect`, which is promoted by high BMI and specific hormonal conditions.

2.2.3 Antigen-Presenting Cells

- **Dendritic Cells (DCs).** Professional antigen-presenting cells that form the critical link between innate and adaptive immunity. In the model, DCs migrate toward tumor cells, phagocytose them (with probability modulated by the `dc_phagocitosis_effect`), and then migrate toward naive T cells ($CD4^+$ or $CD8^+$, chosen randomly at creation) to present the captured antigens. A successful antigen presentation event triggers T cell activation—differentiation of a $CD8^+$ naive cell into a cytotoxic T cell or a $CD4^+$ naive cell into a Th1 or Th2 cell. When no tumor cells are in the neighborhood, DCs employ glucose chemotaxis to navigate toward nutrient-rich regions likely to harbor tumor activity.

- **Plasmacytoid Dendritic Cells (pDCs).** A specialized DC subtype with distinct functional properties. Like conventional DCs, pDCs phagocytose tumor cells and activate T cells. Additionally, pDCs can spawn new NK cells, providing a direct link between antigen presentation and innate cytotoxic capacity. pDCs also exert paracrine effects on their neighborhood: promoting angiogenesis, increasing Treg differentiation, modulating T cell proliferation, and enhancing both T cell and NK cell kill rates.

2.2.4 Macrophages

Tumor-associated macrophages (TAMs) exist along a polarization spectrum, with M1 and M2 representing the two extremes:

- **M1 Macrophages (pro-inflammatory).** M1 macrophages are classically activated and exhibit anti-tumor properties. They phagocytose tumor cells, present antigens to $CD4^+$ naive T cells (similar to DCs), and exert local effects that increase T cell kill rates and promote Th1 proliferation. When no tumor target is available, M1 macrophages use glucose chemotaxis as a fallback navigation strategy. M1 macrophages can polarize to M2 under the influence of the `macrophage_m1_mutation_effect`, which is enhanced by high BMI.

- **M2 Macrophages (anti-inflammatory / pro-tumorigenic).** Alternatively activated M2 macrophages promote tumor progression. They suppress T cell killing (negative `t_kill_rate_effect`), increase tumor growth, and promote angiogenesis. M2 macrophages migrate toward tumor cells and can repolarize to M1 under appropriate conditions, although this transition is less likely than the M1-to-M2 shift in the tumor microenvironment.

The M1/M2 balance is a critical determinant of tumor outcome. A TME dominated by M1 macrophages favors immune-mediated tumor control, while M2 dominance promotes immune evasion and tumor progression. BMI influences this balance through the `macrophage_m1_mutation_effect` and `macrophage_m2_mutation_effect` fields in the effect system, reflecting clinical observations that obesity is associated with increased M2 polarization [7].

2.2.5 Natural Killer Cells

Natural killer (NK) cells are innate lymphocytes capable of recognizing and killing tumor cells without prior antigen sensitization. NK cell cytotoxicity is governed by the balance

between activating and inhibitory receptors on the NK cell surface. In particular, NK cells are inhibited by MHC class I molecules on target cells; tumors that downregulate MHC-I to evade CTL recognition become more susceptible to NK-mediated killing—a phenomenon known as the "missing self" hypothesis.

In the model, NK cell kill probability is determined by the `nkl_kill_rate_effect`, which is influenced by BMI (higher BMI reduces NK effectiveness), neighboring immune cells, and local glucose concentration. NK cells also exert a positive effect on T cell kill rates in their neighborhood, providing paracrine support for adaptive immunity.

2.2.6 Mast Cells and Neutrophils

- **Mast Cells.** These innate immune cells play a dual role in the TME. Depending on stochastic initialization, individual mast cells may promote or suppress tumor growth, enhance or inhibit T cell killing, promote angiogenesis, and modulate M1 macrophage recruitment. Mast cells can also spawn dendritic cells, amplifying the antigen presentation capacity of the TME. This functional heterogeneity—captured in the model by randomly assigning effect polarities at creation—reflects the biological observation that mast cells can be either pro- or anti-tumorigenic depending on context.

- **Neutrophils.** The most abundant innate immune cells in circulation. In the current model, neutrophils are present in the microenvironment but have minimal active behavior, serving primarily as a population marker. Future extensions may incorporate neutrophil-mediated tumor killing and NETosis.

2.2.7 Immune Checkpoints: The PD-1/PD-L1 Pathway

Immune checkpoints are inhibitory pathways that normally prevent autoimmunity but are co-opted by tumors to evade immune destruction. The PD-1/PD-L1 (programmed death-1 / programmed death-ligand 1) axis is the most clinically relevant checkpoint in RCC [11].

PD-L1 (encoded by the *CD274* gene) is expressed on tumor cells and binds to PD-1 on activated T cells, delivering an inhibitory signal that reduces T cell proliferation, cytokine production, and cytotoxic activity. In the model, when a cytotoxic T cell encounters a tumor cell, the tumor cell's `get_PD1_inhibition()` method determines whether PD-1/PD-L1 engagement occurs, based on the *CD274* gene's mutation-expression score. If the checkpoint is engaged, the T cell's activation level is reduced rather than the tumor cell being killed.

2.2.8 Tumor Immune Evasion Mechanisms

RCC tumors employ multiple strategies to evade immune destruction, several of which are captured in the model:

1. **Antigen presentation loss.** Mutations in MHC, B2M, JAK1, JAK2, TAP1, or TAP2 genes reduce the probability of neoantigen presentation, making tumor cells invisible to T cells.
2. **PD-L1 upregulation.** Overexpression of the *CD274* gene product enables immune checkpoint engagement, suppressing T cell activity.

3. **Treg recruitment.** The tumor microenvironment promotes differentiation of $CD4^+$ naive T cells into immunosuppressive Tregs.
4. **M2 macrophage polarization.** Tumor-derived signals (represented through the effect system) promote M1-to-M2 repolarization, shifting the macrophage balance toward pro-tumorigenic phenotypes.
5. **Metabolic competition.** Tumor cells consume glucose at elevated rates (Warburg effect), creating nutrient-depleted zones that impair immune cell function (see Section 2.3).

2.3 Tumor Metabolism

2.3.1 The Warburg Effect

In 1956, Otto Warburg observed that cancer cells preferentially metabolize glucose through glycolysis rather than oxidative phosphorylation, even in the presence of adequate oxygen—a phenomenon now known as aerobic glycolysis or the Warburg effect [14]. While glycolysis is far less efficient than oxidative phosphorylation in terms of ATP yield per glucose molecule (2 versus ∼36 ATP), it provides several advantages to rapidly proliferating tumor cells:

- Faster ATP production rate (glycolysis has a higher flux despite lower yield).
- Generation of biosynthetic precursors (nucleotides, amino acids, lipids) for cell division.
- Acidification of the microenvironment through lactate secretion, which inhibits immune cell function.

In the model, the Warburg effect is represented by a higher glucose consumption rate for tumor cells (`w_glucose_tumor_consumption`) compared to immune cells (`w_glucose_immune_consu` This differential consumption creates local glucose depletion around tumor masses, particularly in poorly vascularized regions.

2.3.2 Metabolic Competition in the TME

The metabolic demands of tumor cells place them in direct competition with immune cells for limited glucose. Activated T cells, in particular, undergo a metabolic switch upon activation, becoming heavily dependent on glycolysis for their effector functions. When glucose is scarce, T cell proliferation, cytokine production, and cytotoxic capacity are all impaired [7].

This metabolic competition creates a positive feedback loop that favors the tumor: as tumor cells consume glucose, immune cells in the depleted zone become less effective, allowing further tumor expansion and greater glucose depletion. Breaking this cycle—for instance, through anti-angiogenic therapy that paradoxically normalizes vasculature and improves nutrient delivery, or through direct metabolic intervention—is an active area of therapeutic research.

2.3.3 Glucose as a Limiting Factor for Immune Function

In the model, glucose availability modulates immune cell behavior through two mechanisms:

1. **Kill probability impairment.** The kill probability of cytotoxic T cells and NK cells is scaled by a glucose factor:

$$p_{\text{kill,effective}} = p_{\text{kill,base}} \times \min\big(1,\ C_{\text{glucose}}(\mathbf{x}) \cdot w_{\text{glucose_immune_sensitivity}}\big) \tag{2.3}$$

   where $C_{\text{glucose}}(\mathbf{x})$ is the local glucose concentration at position $\mathbf{x}$. When glucose is plentiful, the factor saturates at 1; when glucose is depleted, the kill probability is proportionally reduced.

2. **Chemotaxis.** When immune cells (CTLs, M1 macrophages, DCs, NK cells) have no direct target in their neighborhood, they follow the local glucose gradient as a fallback navigation strategy:

$$\Delta\mathbf{x} = \text{sign}\big(\nabla C_{\text{glucose}}(\mathbf{x})\big) \tag{2.4}$$

   This behavior reflects the biological observation that immune cells are attracted to metabolically active regions, and it provides a plausible mechanism for immune cell infiltration toward tumor masses that have not yet been directly detected.

2.4 Treatment Modalities

2.4.1 Immune Checkpoint Inhibitors (ICI)

Immune checkpoint inhibitors (ICIs) are monoclonal antibodies that block inhibitory receptor-ligand interactions on T cells, thereby restoring anti-tumor immune activity. In the context of RCC, the most clinically relevant checkpoint is the PD-1/PD-L1 axis [11].

Anti-PD-1 antibodies (e.g., nivolumab, pembrolizumab) bind to PD-1 on T cells, while anti-PD-L1 antibodies (e.g., avelumab, atezolizumab) bind to PD-L1 on tumor cells. In both cases, the inhibitory signal is blocked, allowing T cells to recognize and kill PD-L1-expressing tumor cells.

In the model, ICI treatment is implemented through three mechanisms, each applied with probability proportional to the drug's effectiveness parameter (`w_ici_effectiveness`):

1. **PD-1/PD-L1 blockade.** The `ICI_effect` flag on tumor cells is set to `True`, causing `get_PD1_inhibition()` to return `False`—the tumor cell can no longer inhibit T cells through the PD-1 pathway.
2. **T cell reactivation.** The `t_activation_effect` of affected T cells is restored to 1.0, counteracting prior exhaustion from checkpoint engagement.
3. **Immune infiltration enhancement.** The `immune_infiltration_factor` of Th1, Th2, and mast cells is incremented, increasing the number of immune cells recruited per spawning event.

2.4.2 Tyrosine Kinase Inhibitors (TKI)

Tyrosine kinase inhibitors (TKIs) target the VEGF receptor signaling pathway, inhibiting angiogenesis and depriving the tumor of its blood supply. In ccRCC, where VEGF overexpression is driven by VHL loss, TKIs have been a cornerstone of treatment [12].

In the model, TKI treatment exerts three effects:

1. **Growth inhibition.** The `tumour_growth_effect` of affected tumor cells is reduced proportionally to the drug's effectiveness.

2. **Anti-angiogenesis.** The `angiogenesis_effect` of affected tumor cells is similarly reduced, decreasing the probability of new blood vessel formation.
3. **Neoantigen release.** Tumor cells with the `TKI_effect` flag set release neoantigens to nearby dendritic cells upon apoptosis, potentially enhancing subsequent adaptive immune responses—a mechanism that provides the immunological rationale for combining TKI with ICI.

Additionally, TKI treatment reduces Treg differentiation by decreasing the `treg_differentiation_e` of $CD4^+$ naive T cells, counteracting one of the tumor's immune evasion strategies.

2.4.3 Combination ICI+TKI Therapy

The combination of ICI and TKI has emerged as the standard first-line treatment for advanced RCC, based on results from landmark clinical trials including KEYNOTE-426 [12] and JAVELIN Renal 101 [10]. The rationale for combination therapy is synergistic: TKI reduces tumor burden and normalizes vasculature (improving immune cell access), while simultaneously releasing neoantigens that prime adaptive immunity; ICI then removes the PD-1/PD-L1 brake, allowing the primed immune system to mount an effective anti-tumor response.

The ARON-1 study [13] extends this paradigm by investigating whether the optimal treatment strategy depends on patient sex, motivated by the observation that sex hormones differentially modulate both the tumor and the immune response. In the model, combination therapy is implemented by applying both `ICIDrug.step()` and `TKIDrug.step()` at each treatment step, with treatment initiation occurring at the step specified by the `treatment_start` parameter.

The model supports four treatment configurations:

- **None**: No drug treatment (control/untreated arm).
- **ICI**: Immune checkpoint inhibitor monotherapy.
- **TKI**: Tyrosine kinase inhibitor monotherapy.
- **ICI+TKI**: Combination immunotherapy and targeted therapy.

This treatment framework, combined with the sex hormone system described in Chapter 7, enables the model to investigate sex-stratified treatment responses—the central question of the ARON-1 study.

Chapter 3

Model Description

This chapter provides a comprehensive description of the RCC agent-based model architecture. We begin with a high-level overview of the simulation environment and time step structure (Section 3.1), then describe the agent class hierarchy (Section 3.2), the effect system that mediates inter-agent communication (Section 3.3), the detailed interaction rules for each agent type (Section 3.4), the movement system (Section 3.5), and the terminal conditions that govern simulation termination (Section 3.6).

3.1 Overview

3.1.1 Spatial Environment

The simulation takes place in a three-dimensional discrete grid of dimensions $W \times H \times D$, where $W = H = D$ and the grid dimension is computed from the patient's tumor volume and a configurable block size parameter:

$$W = \left\lfloor \sqrt[3]{\frac{V_{\text{tumor}}}{V_{\text{block}}}} \right\rfloor \tag{3.1}$$

where V_{tumor} is the tumor volume in mm^3 and V_{block} is the block size parameter.

The grid uses **sticky borders**: agents that attempt to move beyond any grid boundary are clamped to the boundary coordinate. This prevents agents from wrapping around or disappearing, and it creates natural accumulation zones at the grid periphery that can represent tissue boundaries. The grid supports **multiple occupancy**: any number of agents can coexist at the same grid position, reflecting the dense packing of cells in the TME.

A continuous 3D glucose concentration field (described in detail in Chapter 4) is overlaid on the discrete agent grid, sharing the same dimensions. This hybrid discrete-continuous architecture allows the model to capture both individual cell behaviors and the smooth diffusion of nutrients.

3.1.2 Agent Types

The model comprises 18 distinct agent types, each identified by an integer ID in the `AgentType` enumeration. Table 3.1 lists all agent types with their IDs and functional categories.

Table 3.1: Agent types in the RCC model.

ID	Agent Type	Class Name	Category
0	Tumor Cell	`TumorCell`	Neoplastic
1	Cytotoxic T Cell	`CytotoxicTCell`	Adaptive Immune
2	$CD8^+$ Naive T Cell	`CD8NaiveTCell`	Adaptive Immune
3	$CD4^+$ Naive T Cell	`CD4NaiveTCell`	Adaptive Immune
4	$CD4^+$ Th1 Cell	`CD4Helper1TCell`	Adaptive Immune
5	$CD4^+$ Th2 Cell	`CD4Helper2TCell`	Adaptive Immune
6	Regulatory T Cell	`TregCell`	Adaptive Immune
7	Dendritic Cell	`DendriticCell`	Innate Immune (APC)
8	Plasmacytoid DC	`PlasmacitoidDendriticCell`	Innate Immune (APC)
9	M1 Macrophage	`MacrophageM1`	Innate Immune
10	M2 Macrophage	`MacrophageM2`	Innate Immune
11	Natural Killer	`NaturalKiller`	Innate Immune
12	Mast Cell	`MastCell`	Innate Immune
13	Neutrophil	`Neutrophil`	Innate Immune
14	Adipocyte	`Adipocyte`	Stromal
15	Blood Vessel	`Blood`	Vasculature
16	Sex Hormone	`SexHormone`	Signaling
17	Cytokine	`Cytokine`	Signaling

3.1.3 Time Step Structure

Each simulation step follows a fixed sequence of operations, executed by the `RCCModel.step()` method. The ordering is designed to ensure that effects are collected before agents act, that treatment is applied at the correct time, and that spatial updates are deferred to avoid order-dependent artifacts.

1. **Collect data.** Record current population counts, glucose statistics, and kill tallies for all agent types.

2. **Check termination.** Evaluate terminal conditions (Section 3.6). If any condition is met, write the data log and halt.

3. **Apply effects.** For each agent that `needs_effect()`, collect effect objects from neighboring agents within the effect radius and sum them into the agent's `experienced_effects`.

4. **Treatment.** If the current step $t \geq t_{\text{start}}$, apply all active drug effects (ICI, TKI, or both).

5. **Sex drift.** Apply the sex-dependent drift to the $CD8^+$ naive T cell population (duplication for males, removal for females), decrementing the remaining drift steps.

6. **Glucose field step.** Blood vessels inject glucose, diffusion spreads it through the 3D field, and natural decay reduces concentrations.

7. **Manage blood.** Evaluate angiogenesis conditions and create new tumor-generated blood vessels if warranted. Refresh blood access for all tumor cells via BFS from blood-connected sources.

8. **Manage search dimension.** Update the global cell search dimension based on the current tumor cell count, ensuring that immune cells can detect tumors at appropriate ranges.

9. **Shuffle and step all agents.** Take a snapshot of the agent list, shuffle it randomly to eliminate activation order bias, and invoke each agent's `step()` method. Agents may set their `_desired_pos` during this phase but are not yet moved.

10. **Deferred move.** Batch-move all agents whose `_desired_pos` differs from their current position, updating the spatial index.

11. **Spawn hormones.** Generate sex hormone agents at blood vessel locations according to sex-specific hormone chances.

3.2 Agent Hierarchy

The agent class hierarchy reflects the biological taxonomy of the cell types and the shared functionality among related agent types.

3.2.1 Base Cell Class

The `Cell` class (defined in `src/agents/cell.py`) is the abstract base class for all cellular agents. It inherits from both Python's `ABC` and `repast4py.core.Agent`, combining abstract method enforcement with Repast4Py's agent identity management.

Key attributes and methods of `Cell`:

- `pos`: The (x, y, z) position tuple on the 3D grid.
- `size_diameter`: The spatial extent of the cell (default 1).
- `_desired_pos`: The target position for deferred movement.
- `experienced_effects`: An `Effect` object accumulating all effects received from neighbors.
- `search_dimension`: The radius within which the agent searches for targets.
- `_alive`: Boolean flag; once set to `False`, the agent is considered dead and pending removal.
- `base_step()`: Common pre-step logic that applies a base death probability (`w_cell_base_death_p` removing the cell with a small random chance each step.
- `move(new_pos)`: Sets `_desired_pos` without immediately moving the agent.
- `move_towards(target_type, look_up_size, distance)`: Finds the nearest agent of the specified type, computes a normalized direction vector, and sets a deferred move toward it.
- `find_one(agent_type_id, radius)`: Returns a single agent of the given type within the specified Moore neighborhood radius.

- `collect_effects()`: Gathers `Effect` objects from all neighboring `Cell` instances that `has_effect()`.
- `duplicate()`: Creates a copy of the agent in an empty neighboring position.
- `sigmoid(x, k)`: Utility function returning $1/(1 + e^{-kx})$, used for probability normalization.

3.2.2 TCell Base Class

The `TCell` class (defined in `src/agents/t_cell.py`) extends `Cell` with functionality specific to T lymphocytes:

- **Hormone perception.** A dictionary `perceived_sex_hormone` tracks accumulated levels of estrogen, testosterone, progesterone, and androgen. The `perceive_sex_hormone()` method searches the neighborhood for sex hormone agents of the specified type, consumes them (removes from the model), and increments the perceived level. Hormone levels decay by 10% per step via `apply_hormonal_decay()`.
- **TCR receptor matching.** Each T cell carries an 8-bit integer `receptor` value, generated randomly at creation. Antigen recognition is determined by the `is_matching(antigen, receptor)` method, which computes the Hamming distance between the receptor and the SHA-256 hash of the antigen peptide sequence (truncated to 8 bits via `amino_to_8bit_hash()`). A match occurs if:

$$d_H\big(\text{hash(antigen)},\ \text{receptor}\big) \leq 3 + \Delta_{\text{threshold}} \tag{3.2}$$

  where d_H is the Hamming distance and $\Delta_{\text{threshold}}$ is the `receptor_threshold_variation` weight parameter. Additionally, the antigen must not be a self-antigen (i.e., not present in the wildtype protein sequences).
- **Sex hormone modulation.** The method `compute_sexhormone_modulation()` computes composite effects from perceived estrogen, testosterone, and progesterone levels. Each hormone has three regimes (low, medium, high, defined by normalized level thresholds at 0.33 and 0.66) that map to different effect profiles. The effects are weighted by hormone-specific weights (estrogen: 0.45, testosterone: 0.30, progesterone: 0.25) and accumulated in the `accumulated_effects` dictionary, which spans 17 hormonal influence categories including kill rate, proliferation, apoptosis, activation, differentiation, Th1/Th2 proliferation, cytokine production, MHC regulation, costimulation, and checkpoint effects.

3.2.3 Concrete Agent Classes

Each of the 18 agent types is implemented as a concrete class that inherits from either `Cell`, `TCell`, or `repast4py.core.Agent` directly:

- **Inheriting from** `TCell` (and thus `Cell`): `CytotoxicTCell`, `CD8NaiveTCell`, `CD4NaiveTCell`, `CD4Helper1TCell`, `CD4Helper2TCell`.
- **Inheriting from** `Cell` directly: `TumorCell`, `TregCell`, `DendriticCell`, `PlasmacitoidDendritic` `MacrophageM1`, `MacrophageM2`, `NaturalKiller`, `MastCell`, `Neutrophil`, `Adipocyte`, `Blood`.

- **Inheriting from** `repast4py.core.Agent` directly: `SexHormone`, `Cytokine`. These non-cellular agents do not participate in the effect system and have simplified state (position, type-specific attributes, alive flag).

3.3 Effect System

3.3.1 Effect Fields

The effect system is the primary mechanism for paracrine signaling between agents. Each `Cell` agent maintains an `experienced_effects` object of type `Effect` (defined in `src/systems/effect.py`), which contains 14 floating-point fields representing the accumulated influence of neighboring cells:

1. `t_kill_rate_effect` — modulates cytotoxic T cell and NK cell kill probability.
2. `t_proliferation_effect` — modulates T cell proliferation rate.
3. `t_apoptosis_effect` — modulates T cell apoptosis probability.
4. `t_activation_effect` — modulates T cell activation level (affects kill rate and proliferation multiplicatively).
5. `th1_proliferation_effect` — specifically modulates Th1 cell proliferation.
6. `th2_proliferation_effect` — specifically modulates Th2 cell proliferation.
7. `treg_differentiation_effect` — governs $CD4^+$ naive $\rightarrow$ Treg differentiation probability.
8. `macrophage_m1_mutation_effect` — governs M1 $\rightarrow$ M2 macrophage polarization probability.
9. `macrophage_m2_mutation_effect` — governs M2 $\rightarrow$ M1 macrophage repolarization probability.
10. `angiogenesis_effect` — governs the probability of new blood vessel formation.
11. `tumour_growth_effect` — modulates tumor cell duplication probability.
12. `tumour_apoptosis_effect` — modulates tumor cell apoptosis probability.
13. `dc_phagocitosis_effect` — modulates dendritic cell and pDC phagocytosis probability.
14. `nkl_kill_rate_effect` — modulates NK cell kill probability.

3.3.2 BMI Normalization

Three of the 14 effect fields are initialized with BMI-dependent baseline values rather than zero:

- `treg_differentiation_effect`: initialized to $-\hat{B} \times w_{\text{BMI_on_treg_diff}}$
- `macrophage_m1_mutation_effect`: initialized to $-\hat{B} \times w_{\text{BMI_on_m1_mutation}}$
- `macrophage_m2_mutation_effect`: initialized to $+\hat{B} \times w_{\text{BMI_on_m2_mutation}}$
- `nkl_kill_rate_effect`: initialized to $-\hat{B} \times w_{\text{BMI_nkl_kill_rate}}$

where the normalized BMI is:

$$\hat{B} = \frac{\text{BMI} - \mu_{\text{BMI}}}{\sigma_{\text{BMI}}} \tag{3.3}$$

with sex-specific parameters $\mu_{\text{BMI}} = 26.5$, $\sigma_{\text{BMI}} = 5.5$ for females and $\mu_{\text{BMI}} = 27.5$, $\sigma_{\text{BMI}} = 4.5$ for males. This normalization ensures that a BMI above the population mean increases pro-tumorigenic effects (M2 polarization, Treg differentiation) and decreases anti-tumorigenic effects (NK killing, M1 stability), consistent with clinical observations linking obesity to poorer RCC outcomes.

3.3.3 Effect Collection and Aggregation

At each simulation step, before agents execute their individual behaviors, the model's `apply_effects()` method iterates over all agents. For each agent that returns `True` from `needs_effect()`, the `collect_and_apply_effects()` method:

1. Queries the Moore neighborhood (radius 1) via the spatial index.
2. For each neighbor that is a `Cell` instance and returns `True` from `has_effect()`, retrieves its `Effect` object via `get_effect()`.
3. Aggregates all collected effects with the agent's current `experienced_effects` via the `sum_with()` method, which performs element-wise addition across all 14 fields.

The `Effect.EFFECT_RADIUS` class variable is set to 3, although the actual collection radius in the current implementation is 1 (Moore neighborhood).

3.4 Agent Interactions

Table 3.2 summarizes the key interactions between agent types. Each row describes an agent type's effects on other agents, organized by the mechanism of interaction.

Table 3.2: Summary of agent interactions in the RCC model.

Agent Type	Mechanism	Effects on Other Agents
`TumorCell`	Effect emission	Emits positive `angiogenesis_effect`.
	PD-1/PD-L1	Inhibits CTL activation via `get_PD1_inhibition()` based on *CD274* gene expression. Blocked by ICI treatment.
	Duplication	Divides into empty neighbor; probability modulated by `tumour_growth_effect`, DNA proliferation genes, angiogenesis, blood access, and local glucose.
	Apoptosis	Dies based on `tumour_apoptosis_effect` and DNA tumor suppressor genes. On death with TKI, releases neoantigens to nearby DCs.

Continued on next page

Agent Type	Mechanism	Effects on Other Agents
	Glucose	Consumes glucose at rate `w_glucose_tumor_consumption` (Warburg effect). Growth scaled by local glucose via `w_glucose_growth_sensitivity`.
`CytotoxicTCell`	Killing	Kills adjacent tumor cells with probability based on `t_kill_rate_effect` × `t_activation_effect`, impaired by low glucose. Blocked by PD-1/PD-L1 engagement.
	Exhaustion	Loses `t_activation_effect` progressively with each kill (`w_progressive_exhaustion`) or sharply upon PD-1 inhibition.
	Proliferation	Duplicates based on proliferation chance × activation level.
	Hormone response	Perceives estrogen, progesterone, testosterone; estrogen enhances kill rate and proliferation, testosterone suppresses.
`MacrophageM1`	Effect emission	Positive `t_kill_rate_effect` and `th1_proliferation_effect`.
	Phagocytosis	Engulfs adjacent tumor cells; after phagocytosis, migrates to and activates $CD4^+$ naive T cells.
	Polarization	Can transform to M2 based on `macrophage_m1_mutation_effect`.
	Glucose chemotaxis	Follows glucose gradient when no tumor target found.
`MacrophageM2`	Effect emission	Negative `t_kill_rate_effect`; positive `tumour_growth_effect` and `angiogenesis_effect`.
	Polarization	Can transform to M1 based on `macrophage_m2_mutation_effect`.
`TregCell`	Effect emission	Strong negative `t_kill_rate_effect` (-0.3); negative `t_proliferation_effect`, `t_activation_effect`, `dc_phagocitosis_effect`; positive `t_apoptosis_effect`.
`DendriticCell`	Phagocytosis	Engulfs adjacent tumor cells based on `dc_phagocitosis_effect`.

Continued on next page

Agent Type	Mechanism	Effects on Other Agents
	T cell activation	After phagocytosis, migrates to $CD4^+$ or $CD8^+$ naive T cells and presents captured antigen, triggering activation via TCR matching.
	Glucose chemotaxis	Follows glucose gradient when no tumor target found.
`PlasmacitoidDC`	Effect emission	Positive `angiogenesis_effect`, `treg_differentiation_effect`, `t_kill_rate_effect`, `nkl_kill_rate_effect`; negative `t_proliferation_effect`.
	Phagocytosis	Same as DC: engulfs tumors, activates naive T cells.
	NK spawning	Spawns NK cells in empty neighbors with probability `spawn_nkl_chance`.
`NaturalKiller`	Killing	Kills adjacent tumor cells based on `nkl_kill_rate_effect`, impaired by low glucose.
	Effect emission	Positive `t_kill_rate_effect`.
	Glucose chemotaxis	Follows glucose gradient when no tumor target found.
`MastCell`	Effect emission	Stochastic combination of effects (determined at creation): `angiogenesis_effect` (±), `macrophage_m1_mutation_effect` (±), `t_kill_rate_effect` (±), `tumour_apoptosis_effect` (±), `tumour_growth_effect` (±).
	DC spawning	Spawns dendritic cells at border positions with probability `spawn_dc_chance`.
`CD4NaiveTCell`	Treg differentiation	Differentiates into Treg based on `treg_differentiation_effect`, sex hormones (estrogen, progesterone, testosterone), and a sigmoidal gene factor.
	Activation	When presented with a matching antigen by a DC or M1 macrophage, differentiates into Th1 or Th2 with equal probability.

Continued on next page

Agent Type	Mechanism	Effects on Other Agents
`CD8NaiveTCell`	Activation	When presented with a matching antigen, differentiates into CytotoxicTCell. Targeted by sex drift mechanism (duplication in males, removal in females).
`CD4Helper1TCell`	Effect emission Recruitment	Positive `tumour_apoptosis_effect`. Spawns M1 macrophages and DCs at grid border positions. Proliferates based on `th1_proliferation_effect` × activation level, modulated by estrogen.
`CD4Helper2TCell`	Effect emission Recruitment	Positive `t_kill_rate_effect` and `tumour_growth_effect`. Spawns M1 macrophages and CytotoxicTCells. Proliferates based on `th2_proliferation_effect` × activation level, enhanced by estrogen and progesterone.
`Blood`	Glucose source	Blood vessels inject glucose into the field at each step. Tumor-generated vessels (`tumour_generated=True`) are distinguished from pre-existing ones for angiogenesis logic.
`SexHormone`	Hormone delivery	Diffuses through the grid via biased random walk (away from blood vessels). Consumed by T cells in the neighborhood, modulating their immune behavior. Removed upon consumption or when leaving the grid.

3.5 Movement

3.5.1 Deferred Movement Pattern

To prevent order-dependent artifacts—where an agent that moves early in the step occupies a position that a later agent expects to be empty—the model uses a deferred movement pattern:

1. During their `step()` execution, agents call `self.move(new_pos)`, which sets `self._desired_pos = new_pos` without modifying the spatial index.
2. After all agents have stepped, the model's `move_all_agents()` method iterates over all living `Cell` agents and, for each agent whose `_desired_pos` differs from `pos`, updates the spatial index and grid position.

This two-phase approach ensures that all agents make movement decisions based on the same spatial snapshot, and it eliminates bias arising from the random activation order.

3.5.2 Directed Movement: `move_towards()`

The primary movement mechanism for most agents is `move_towards(target_agent_type_id, look_up_size, distance)`, which:

1. Retrieves all living agents of the target type.
2. Finds the closest target by Manhattan distance:

$$d_M(\mathbf{a}, \mathbf{b}) = |a_x - b_x| + |a_y - b_y| + |a_z - b_z| \tag{3.4}$$

3. If the closest target is within `look_up_size`, computes the Euclidean direction vector $\mathbf{d} = \mathbf{p}_{\text{target}} - \mathbf{p}_{\text{self}}$, normalizes it, and scales by `distance`.
4. Rounds each component and clamps to grid boundaries (sticky borders).
5. Sets the resulting position as `_desired_pos`.

The `look_up_size` parameter is typically derived from the model's `cell_search_dimension`, which grows with the tumor cell count to allow immune cells to detect tumors at greater distances as the tumor mass expands.

3.5.3 Glucose Chemotaxis

When an immune cell (CTL, M1 macrophage, DC, or NK cell) fails to find a direct target via `move_towards()`, it may fall back to glucose chemotaxis. With probability `w_glucose_chemotaxis_strength`, the agent queries the glucose field for the discretized gradient at its position:

$$(\Delta x, \Delta y, \Delta z) = \text{sign}\left(\nabla C_{\text{glucose}}(\mathbf{x})\right) \tag{3.5}$$

where the gradient is computed via central finite differences with Neumann (zero-flux) boundary conditions. The agent then sets its desired position to $\mathbf{x} + (\Delta x, \Delta y, \Delta z)$, clamped to grid bounds. This mechanism causes immune cells to migrate toward glucose-rich regions—typically areas near blood vessels and actively growing tumor masses—providing a biologically plausible navigation strategy in the absence of direct antigen detection.

3.5.4 Sex Hormone Diffusion

Sex hormone agents follow a distinct movement model. At each step, a hormone agent randomly selects one of two diffusion strategies (each with 50% probability):

- **Biased diffusion.** The hormone evaluates all 26 Moore neighbors, computes the Manhattan distance from each to the nearest blood vessel, and selects a target position via weighted random choice. The weighting biases movement away from blood vessels (where hormones are spawned), promoting spatial distribution through the TME. The bias strength is controlled by a `drift_strength` parameter set to 0.25.

- **Simple diffusion.** The hormone selects a random move from a predefined set of directions biased toward positive y and z coordinates, or from the two axis-aligned directions $(0, 0, 1)$ and $(0, 1, 0)$. If the resulting position falls outside the grid, the hormone is removed from the simulation.

3.6 Terminal Conditions

The simulation terminates when any of the following conditions is met. The `terminal_condition()` method returns a tuple `(terminated, survival)` indicating whether the simulation should stop and whether the outcome represents patient survival (tumor elimination).

1. **Maximum steps reached.** If $t \geq t_{\max}$ (`max_steps`), the simulation terminates with `survival = False`. This represents the end of the observation period without definitive tumor clearance.

2. **Tumor elimination.** If the tumor cell count drops to zero before reaching `max_steps`, the simulation terminates with `survival = True`. This outcome indicates that the immune system (with or without treatment) successfully eliminated all tumor cells.

3. **Tumor cell count exceeds threshold.** If the number of living tumor cells reaches or exceeds `max_tumor_cells` (default: 2000), the simulation terminates with `survival = False`. This condition represents uncontrolled tumor growth that has exceeded the model's capacity and, clinically, would correspond to a large, aggressive tumor mass.

4. **Mean tumor growth rate exceeds threshold.** The mean tumor growth rate across all living tumor cells is computed as:

$$\bar{G} = \frac{1}{N_T} \sum_{i=1}^{N_T} \sigma\big(g_i + a_i,\ k = 10\big) \tag{3.6}$$

   where N_T is the tumor cell count, g_i is the growth potential (combining `tumour_growth_effect`, DNA-derived proliferation chance, blood access, and angiogenesis), a_i is the apoptosis-related component, and $\sigma(\cdot)$ is the sigmoid function with steepness $k = 10$. If $\bar{G} \geq w_{\text{tumour_growth_threshold}}$, the simulation terminates with `survival = False`. This condition detects cases where individual tumor growth rates are high even if the absolute cell count has not yet reached the threshold, capturing scenarios of rapidly accelerating tumor progression.

These terminal conditions collectively ensure that simulations reach a definitive outcome—either tumor elimination or tumor dominance—within a bounded number of steps, while capturing both the absolute size and the growth dynamics of the tumor population.

Chapter 4

Glucose Microenvironment System

This chapter describes the glucose concentration field that overlays the discrete agent grid, modeling metabolic dynamics within the tumor microenvironment. The system captures diffusion, source injection from blood vessels, consumption by tumor and immune cells, and gradient-based chemotaxis for immune cell navigation.

4.1 Biological Motivation

The metabolic landscape of the tumor microenvironment plays a decisive role in determining the outcome of immune–tumor interactions. Several biological phenomena motivate the inclusion of an explicit glucose field in the model.

The Warburg Effect. In the 1920s, Otto Warburg observed that tumor cells preferentially metabolize glucose through aerobic glycolysis—even in the presence of sufficient oxygen for oxidative phosphorylation [14]. This metabolic reprogramming, now recognized as a hallmark of cancer [7], results in dramatically elevated glucose uptake by tumor cells (up to 200-fold compared to normal tissue) and the production of large quantities of lactate. The Warburg effect is not merely a metabolic curiosity; it confers a selective advantage by generating biosynthetic precursors for rapid proliferation and by acidifying the local microenvironment, which impairs immune cell function.

Metabolic Competition. Because tumor cells consume glucose at a greatly accelerated rate, the regions immediately surrounding the tumor mass become nutrient-deprived. This metabolic competition between tumor cells and infiltrating immune cells creates nutrient gradients that shape the spatial organization of the microenvironment. Immune cells arriving at the tumor periphery may encounter glucose concentrations insufficient to sustain their effector functions.

Impact on Immune Function. T cells and Natural Killer (NK) cells are critically dependent on glucose availability for their cytotoxic activity. Upon activation, T cells undergo a metabolic switch to aerobic glycolysis to support clonal expansion and effector function. When glucose is scarce, T cell proliferation is impaired, interferon-gamma (IFN-γ) production decreases, and cytotoxic granule release is diminished. Similarly, NK cell cytotoxicity is reduced under glucose-deprived conditions. In the model, this dependency is captured by a glucose-dependent scaling factor applied to immune cell kill rates.

Glucose Gradients as Chemotactic Signals. In the absence of direct cell–cell contact with a target, immune cells can follow glucose concentration gradients as a proxy for proximity to blood vessels and viable tissue. Regions of high glucose concentration tend to be near functional vasculature, while depleted regions indicate areas of heavy tumor metabolism. The model exploits this gradient information to provide a chemotactic fallback behavior for immune cells that have not yet located a tumor target.

4.2 Mathematical Formulation

The glucose concentration field $C(\mathbf{x}, t)$ evolves according to a reaction–diffusion equation derived from Fick's second law [5]:

$$\frac{\partial C}{\partial t} = D\,\nabla^2 C \;-\; \lambda\, C \;+\; S(\mathbf{x}) \;-\; Q(\mathbf{x}), \tag{4.1}$$

where:

- $C(\mathbf{x}, t)$ is the glucose concentration at position $\mathbf{x} = (x, y, z)$ and time t,
- D is the diffusion coefficient (`w_glucose_diffusion`, default 0.1),
- λ is the natural decay rate (`w_glucose_decay`, default 0.01),
- $S(\mathbf{x})$ is the source term (glucose injection from blood vessels),
- $Q(\mathbf{x})$ is the consumption term (uptake by tumor and immune cells).

4.2.1 Discrete 3D Laplacian

The continuous Laplacian $\nabla^2 C$ is discretized on a regular 3D Cartesian grid using the standard 6-connected stencil. For a grid point (i, j, k) with unit spacing $\Delta x = 1$:

$$\begin{aligned}\nabla^2 C(i,j,k) = {} & C(i{+}1,j,k) + C(i{-}1,j,k) \\ & + C(i,j{+}1,k) + C(i,j{-}1,k) \\ & + C(i,j,k{+}1) + C(i,j,k{-}1) \\ & - 6\,C(i,j,k).\end{aligned} \tag{4.2}$$

This discrete Laplacian is implemented as a 3D convolution using `scipy.ndimage.convolve` with a $3 \times 3 \times 3$ kernel K defined by:

$$K[1,1,1] = -6, \qquad K[\text{face neighbors}] = 1, \qquad K[\text{all others}] = 0, \tag{4.3}$$

where the six face-neighbor entries correspond to the positions $(0,1,1)$, $(2,1,1)$, $(1,0,1)$, $(1,2,1)$, $(1,1,0)$, and $(1,1,2)$ in the kernel array. All remaining 20 entries (edges and corners) are zero.

4.2.2 Time Integration

The diffusion equation is advanced in time using explicit (forward) Euler integration:

$$C^{n+1}(i,j,k) = C^n(i,j,k) + D \cdot \nabla^2 C^n(i,j,k). \tag{4.4}$$

4.2.3 Stability Analysis

By the von Neumann stability criterion for the explicit Euler scheme applied to the 3D diffusion equation on a unit grid, the stability condition is:

$$D < \frac{1}{6} \approx 0.1\overline{6}, \tag{4.5}$$

which ensures that the amplification factor remains bounded. The default value $D = 0.1$ satisfies this constraint. After each diffusion step, the field is clamped to non-negative values to prevent unphysical negative concentrations.

4.2.4 Boundary Conditions

Neumann (zero-flux) boundary conditions are enforced by passing `mode='nearest'` to `scipy.ndimage.convolve`. This mode replicates the boundary voxel values, effectively setting $\partial C/\partial \mathbf{n} = 0$ at all domain boundaries, which corresponds to no glucose flux through the walls of the simulation volume.

4.3 Source Model

Glucose enters the simulation domain through blood vessel agents, which act as point sources.

Injection Mechanism. At each simulation tick, every `Blood` agent injects glucose at its grid position:

$$C(\mathbf{x}_v) \leftarrow C(\mathbf{x}_v) + r_{\text{source}}, \tag{4.6}$$

where $\mathbf{x}_v$ is the position of blood vessel agent v and r_{source} is the injection rate per step (`w_glucose_source_rate`, default 5.0).

Blood Vessel Geometry. Blood vessels are initialized along one edge of the simulation cube in a 4×4 cross-sectional arrangement (minus the four corner positions, yielding 12 vessel positions per slice along the x-axis). This geometry creates a vascular channel that serves as the primary glucose supply for the entire domain. Additional tumor-generated blood vessels (via angiogenesis) contribute further glucose sources during the simulation.

4.4 Consumption Model

Glucose is consumed by both tumor cells and immune cells, with consumption handled individually during each agent's `step()` method.

4.4.1 Tumor Cell Consumption

Tumor cells consume glucose at a rate reflecting the Warburg effect:

$$C(\mathbf{x}_t) \leftarrow \max\big(0,\ C(\mathbf{x}_t) - r_{\text{tumor}}\big), \tag{4.7}$$

where $r_{\text{tumor}} =$ `w_glucose_tumor_consumption` (default 2.0 per step). The clamping to zero ensures that glucose concentrations remain non-negative.

4.4.2 Immune Cell Consumption

All immune cells (cytotoxic T cells, NK cells, macrophages, dendritic cells) consume glucose at a lower rate:

$$C(\mathbf{x}_i) \leftarrow \max\big(0,\ C(\mathbf{x}_i) - r_{\text{immune}}\big), \tag{4.8}$$

where $r_{\text{immune}} =$ `w_glucose_immune_consumption` (default 0.5 per step).

4.4.3 Glucose-Dependent Growth Factor

Tumor cell proliferation probability is modulated by local glucose availability:

$$f_{\text{glucose}} = \min\big(1.0,\ C(\mathbf{x}_t) \cdot w_{\text{growth_sensitivity}}\big), \tag{4.9}$$

where $w_{\text{growth_sensitivity}} =$ `w_glucose_growth_sensitivity` (default 1.0). The duplication probability is then scaled by this factor:

$$p'_{\text{dup}} = p_{\text{dup}} \cdot f_{\text{glucose}}. \tag{4.10}$$

When glucose is abundant ($C \geq 1.0$), growth proceeds at the full rate. As glucose drops below 1.0, growth is proportionally suppressed.

4.4.4 Glucose-Dependent Immune Function

Immune cell kill rates are similarly modulated:

$$f_{\text{immune}} = \min\big(1.0,\ C(\mathbf{x}_i) \cdot w_{\text{immune_sensitivity}}\big), \tag{4.11}$$

where $w_{\text{immune_sensitivity}} =$ `w_glucose_immune_sensitivity` (default 1.0). The effective kill probability becomes:

$$p'_{\text{kill}} = p_{\text{kill}} \cdot f_{\text{immune}}. \tag{4.12}$$

This mechanism captures the biological observation that T cells and NK cells require glucose to sustain their cytotoxic machinery.

4.5 Gradient and Chemotaxis

4.5.1 Gradient Computation

The glucose gradient at position (i, j, k) is computed using central finite differences:

$$\begin{aligned}
\left.\frac{\partial C}{\partial x}\right|_{(i,j,k)} &= \frac{C(i{+}1, j, k) - C(i{-}1, j, k)}{2}, \\
\left.\frac{\partial C}{\partial y}\right|_{(i,j,k)} &= \frac{C(i, j{+}1, k) - C(i, j{-}1, k)}{2}, \\
\left.\frac{\partial C}{\partial z}\right|_{(i,j,k)} &= \frac{C(i, j, k{+}1) - C(i, j, k{-}1)}{2}.
\end{aligned} \tag{4.13}$$

At domain boundaries, the indices are clamped to the valid range (consistent with the Neumann boundary conditions), so the gradient naturally approaches zero near the walls.

4.5.2 Discretized Gradient Step

For agent movement on the integer grid, the continuous gradient is converted to a discrete step direction:

$$(\Delta x, \Delta y, \Delta z) = \big(\text{sign}(\nabla_x C),\ \text{sign}(\nabla_y C),\ \text{sign}(\nabla_z C)\big), \tag{4.14}$$

where $\text{sign}(v) \in \{-1, 0, +1\}$. The resulting step is further clamped to ensure the target position lies within the grid bounds.

4.5.3 Chemotaxis Behavior

Immune cells that implement glucose chemotaxis—specifically `CytotoxicTCell`, `MacrophageM1`, `DendriticCell`, and `NaturalKiller`—employ the following fallback strategy:

1. The agent first attempts to move directly towards a nearby tumor cell using the standard `move_towards()` method.
2. If no tumor target is found (i.e., `move_towards()` returns `False`), the agent evaluates the glucose gradient at its current position.
3. With probability p_{chemo} = `w_glucose_chemotaxis_strength` (default 0.3), the agent moves one step in the direction of increasing glucose concentration.
4. If the gradient is zero (flat concentration), no movement occurs.

This mechanism provides immune cells with a biologically motivated navigation strategy: in the absence of direct target detection, cells move towards areas of higher metabolic activity, which correlates with the presence of functional vasculature and, consequently, viable tissue where tumor cells may reside.

4.6 Implementation

The glucose system is encapsulated in the `GlucoseField` class, located in `src/systems/glucose_field.`

4.6.1 Data Structure

The glucose field is stored as a three-dimensional NumPy array of shape (`width`×`height`×`depth`) with `float64` precision, matching the dimensions of the agent grid. The initial concentration is set uniformly to 1.0.

4.6.2 Convolution Kernel

The 6-connected Laplacian kernel is constructed once during initialization as a $3 \times 3 \times 3$ NumPy array:

```
self._kernel = np.zeros((3, 3, 3), dtype=np.float64)
self._kernel[1, 1, 0] = 1.0    # z-1
self._kernel[1, 1, 2] = 1.0    # z+1
self._kernel[1, 0, 1] = 1.0    # y-1
self._kernel[1, 2, 1] = 1.0    # y+1
```

```
self._kernel[0, 1, 1] = 1.0    # x-1
self._kernel[2, 1, 1] = 1.0    # x+1
self._kernel[1, 1, 1] = -6.0   # center
```

Listing 4.1: Laplacian kernel construction.

4.6.3 Step Function

The `step(model)` method executes the three phases of the glucose dynamics in sequence:

1. **Injection:** iterate over all `Blood` agents and call `inject(pos, rate)` for each.
2. **Diffusion:** apply the discrete Laplacian convolution and update the field via explicit Euler.
3. **Decay:** multiply the entire field by $(1 - \lambda)$ where λ is the decay rate.

Agent-level consumption (by tumor and immune cells) is handled within each agent's own `step()` method, not within the `GlucoseField.step()` call. This design ensures that consumption occurs at the correct point in the agent execution order and avoids double-counting.

4.6.4 Statistics Methods

The `GlucoseField` class exposes four aggregate statistics, used by the data collection system to track metabolic dynamics over time:

- `mean_concentration()` – mean glucose across all voxels,
- `total_concentration()` – total glucose in the domain,
- `min_concentration()` – minimum glucose (identifies depletion zones),
- `max_concentration()` – maximum glucose (identifies source neighborhoods).

These values are recorded at every simulation step and provide insight into the metabolic state of the microenvironment, including the formation and extent of glucose-deprived regions around the tumor mass.

Chapter 5

DNA and Mutation System

This chapter describes the gene-level representation used by tumor cells in the model. The DNA system encodes 16 cancer-relevant genes, supports codon-level translation, introduces mutations during cell division with genomic-instability-dependent rates, and derives phenotypic parameters that govern tumor behavior and immune evasion.

5.1 Gene Model

Each tumor cell carries a `DNA` object that encodes a synthetic DNA sequence containing 16 genes chosen for their relevance to renal cell carcinoma immunology. The genes span the major biological axes of antigen presentation, immune checkpoint signaling, DNA repair, tumor suppression, and oncogenic proliferation.

5.1.1 Gene Catalogue

Table 5.1 lists all 16 genes, their functional categories, and their roles in the model.

Table 5.1: The 16 genes modeled in the DNA system and their functional roles.

Gene	Category	Functional Role
MHC	Antigen Presentation	Major histocompatibility complex; presents peptide antigens on the cell surface for T cell recognition.
B2M	Antigen Presentation	Beta-2-microglobulin; essential component of MHC class I molecules.
TAP1	Antigen Presentation	Transporter associated with antigen processing 1; loads peptides onto MHC I.
TAP2	Antigen Presentation	Transporter associated with antigen processing 2; works with TAP1 for peptide loading.
JAK1	Antigen Presentation	Janus kinase 1; IFN-γ signaling for MHC upregulation.
JAK2	Antigen Presentation	Janus kinase 2; cooperates with JAK1 in interferon signaling.

Continued on next page

Gene	Category	Functional Role
CD274	Immune Checkpoint	PD-L1; binds PD-1 on T cells to induce immune checkpoint inhibition.
BRCA1	DNA Repair	Breast cancer type 1 susceptibility protein; key DNA double-strand break repair gene controlling genomic stability.
TP53	Tumor Suppression	Tumor protein p53; "guardian of the genome," triggers apoptosis in damaged cells.
RB1	Tumor Suppression	Retinoblastoma protein; cell cycle checkpoint regulator.
CDKN2A	Tumor Suppression	Cyclin-dependent kinase inhibitor 2A (p16); inhibits CDK4/6 to arrest cell cycle.
MYC	Oncogene	Transcription factor promoting cell growth and proliferation.
RAS	Oncogene	GTPase signal transducer in the MAPK proliferation pathway.
EGFR	Oncogene	Epidermal growth factor receptor; receptor tyrosine kinase driving proliferation.
HER2	Oncogene	Human epidermal growth factor receptor 2; amplified in aggressive tumors.
APC	Tumor Suppression	Adenomatous polyposis coli; Wnt pathway regulator, loss promotes proliferation.

5.1.2 DNA Sequence Structure

Each gene occupies a defined region within a synthetic DNA sequence. The gene metadata specifies, for each gene: a start position (offset in the full DNA string), a coding region length of 90 base pairs, and a wildtype protein sequence of 20 amino acids. The full DNA sequence is assembled during initialization by concatenating, for each gene:

1. A random nucleotide spacer (100 bp),
2. A promoter region: 13 random bases, the `TATA` box motif, and 13 more random bases (total 30 bp),
3. The coding region: codon-encoded wildtype protein (60 bp for 20 amino acids),
4. A stop codon (`TAA`),
5. A trailing spacer (7 bp).

The start positions are fixed (at offsets 100, 300, 500, . . . , 3100 for the 16 genes), ensuring deterministic gene location across all tumor cell lineages. The wildtype protein sequences are fixed 20-character strings drawn from the standard amino acid alphabet.

5.2 Codon Translation

The model implements the standard genetic code, mapping all 64 possible nucleotide triplets to the 20 amino acids and the stop signal.

5.2.1 Codon Table

The codon table is stored as a class-level dictionary in the `DNA` class, with keys being three-letter nucleotide strings (e.g., `ATG` → `M` for methionine) and the three stop codons (`TAA`, `TAG`, `TGA`) mapping to the sentinel character `_`.

5.2.2 Translation: `gene_to_protein()`

Given a gene name, the `gene_to_protein()` method reads codons starting from the gene's start position, translating each triplet to its corresponding amino acid. Translation terminates upon encountering a stop codon:

```python
def gene_to_protein(self, gene_name: str) -> str:
    protein = ""
    start = DNA.gene_metadata[gene_name]["start"]
    for i in range(start, len(self.dna) - 2, 3):
        codon = self.dna[i:i + 3]
        aa = self.codon_table.get(codon, 'X')
        if aa == '_':
            break
        protein += aa
    return protein
```

Listing 5.1: Gene-to-protein translation.

Unknown codons (which should not arise in practice) are mapped to `X`.

5.2.3 Reverse Translation: `protein_to_gene()`

For constructing the initial wildtype DNA sequence, the `protein_to_gene()` static method performs reverse translation. It uses an inverted codon table (amino acid → codon) to convert each amino acid back to a nucleotide triplet. Since the genetic code is degenerate (multiple codons per amino acid), the inverse mapping selects the last codon encountered during dictionary inversion, yielding a deterministic but arbitrary choice among synonymous codons.

5.3 Mutation Mechanics

Mutations accumulate through two mechanisms: injected mutations at initialization and stochastic point mutations during DNA duplication.

5.3.1 Genomic Instability

The mutation rate during cell division is controlled by the genomic instability parameter, which is derived from the BRCA1 gene's mutation status:

$$\mu_{\text{instability}} = \left(s_{\text{BRCA1}}\right)^{\max(1,\, e_{\text{BRCA1}})} \times 0.05, \tag{5.1}$$

where s_{BRCA1} is the BRCA1 mutation score (see Section 5.4.2) and e_{BRCA1} is the BRCA1 promoter expression level. A wildtype BRCA1 ($s_{\text{BRCA1}} = 0$) yields zero instability, while a fully mutated BRCA1 ($s_{\text{BRCA1}} \to 1$) produces a base mutation rate of 0.05 per nucleotide.

The exponentiation by the expression level amplifies the instability when the BRCA1 promoter is active.

5.3.2 DNA Duplication with Point Mutations

When a tumor cell divides, its DNA is duplicated with potential point mutations:

$$n_{\text{mut}} = \lfloor \mu_{\text{instability}} \times L \rfloor, \tag{5.2}$$

where L is the total length of the DNA sequence. For each of the n_{mut} mutations, a random position is selected, and the nucleotide at that position is replaced by one of the three alternative bases (chosen uniformly at random). This process models single-nucleotide variants (SNVs), the most common form of somatic mutation.

5.3.3 Injected Mutations

The initial cancer cell is seeded with pre-existing BRCA1 mutations to establish genomic instability from the outset. The number of injected mutations depends on the patient's sex:

$$n_{\text{injected}} = \begin{cases} 3 & \text{if sex} = \text{Male}, \\ 1 & \text{if sex} = \text{Female}. \end{cases} \tag{5.3}$$

These mutations are introduced by replacing a segment of the BRCA1 coding region (positions 730–790 in the DNA string) with a codon-encoded mutant protein that differs from the wildtype by exactly n_{injected} amino acid substitutions. The substitution positions are selected randomly, and each is replaced by a random alternative from the amino acid alphabet.

5.4 Gene Expression and Phenotypic Effects

Each gene's contribution to tumor phenotype is determined by two quantities: promoter expression and mutation score.

5.4.1 Promoter Expression

Gene expression is modeled as a count of `TATA` motifs in the 30-nucleotide promoter region upstream of the coding sequence:

$$e_g = \#\{\texttt{TATA} \text{ occurrences in promoter region of gene } g\}. \tag{5.4}$$

In the wildtype sequence, each gene has exactly one `TATA` box ($e_g = 1$). Mutations in the promoter region can destroy or create additional `TATA` motifs, thereby altering expression.

5.4.2 Mutation Score

The mutation score quantifies how far the translated protein has diverged from the wildtype:

$$s_g = 1 - \text{Levenshtein.ratio}\big(p_g^{\text{wt}},\ p_g^{\text{mut}}\big), \tag{5.5}$$

where p_g^{wt} is the wildtype protein sequence, p_g^{mut} is the mutant protein, and Levenshtein.ratio returns the normalized similarity (1.0 for identical strings, 0.0 for completely different strings). A mutation score of $s_g = 0$ indicates a wildtype protein, while $s_g = 1$ indicates maximal divergence.

5.4.3 Gene Mutation–Expression Product

Several phenotypic parameters use the product of expression and mutation score:

$$\phi_g = e_g \cdot s_g, \tag{5.6}$$

which captures the idea that a gene must be both expressed and mutated to exert a phenotypic effect.

5.4.4 Five Derived Phenotypic Parameters

The DNA system computes five aggregate phenotypic parameters from the 16 individual gene states. These parameters directly influence tumor cell behavior during simulation.

1. Antigen Presentation Chance.

$$p_{\text{AP}} = \max\left(0,\ 6 - \frac{\sum_{g \in \mathcal{G}_{\text{AP}}} \phi_g}{6}\right), \tag{5.7}$$

where $\mathcal{G}_{\text{AP}} = \{\text{MHC, JAK1, JAK2, B2M, TAP1, TAP2}\}$. This represents the tumor cell's ability to present antigens on its surface. Mutations in antigen presentation genes reduce this probability, enabling immune evasion.

2. Checkpoint Pathway Inhibition Chance.

$$p_{\text{CPI}} = \phi_{\text{CD274}}, \tag{5.8}$$

determined solely by the CD274 (PD-L1) gene. Higher expression of mutated PD-L1 increases the probability that the tumor cell will engage the PD-1 checkpoint on interacting T cells, suppressing their cytotoxic activity.

3. Genomic Instability.

$$\mu_{\text{instability}} = \left(s_{\text{BRCA1}}\right)^{\max(1,\, e_{\text{BRCA1}})} \times 0.05, \tag{5.9}$$

as described in Section 5.3. This parameter controls the mutation rate during DNA duplication.

4. Tumor Suppression Chance.

$$p_{\text{TS}} = \frac{1}{3} \sum_{g \in \mathcal{G}_{\text{TS}}} e_g \cdot (1 - s_g), \tag{5.10}$$

where $\mathcal{G}_{\text{TS}} = \{\text{TP53, RB1, CDKN2A}\}$. Functional (unmutated, expressed) tumor suppressors increase the probability of apoptosis. Loss-of-function mutations ($s_g \to 1$) diminish this protective effect.

5. Extra Proliferation Chance.

$$p_{\mathrm{EP}} = \frac{1}{5}\left(\sum_{g\in\mathcal{G}_{\mathrm{onco}}} e_g \cdot (1 - s_g) + \phi_{\mathrm{APC}}\right), \tag{5.11}$$

where $\mathcal{G}_{\mathrm{onco}} = \{\mathrm{MYC, RAS, EGFR, HER2}\}$. This parameter captures oncogene-driven proliferation. Functional oncogenes and mutated APC (a tumor suppressor whose loss promotes proliferation via the Wnt pathway) both contribute to increased growth. The raw value is converted to a probability using a saturating transformation:

$$\hat{p}_{\mathrm{EP}} = 1 - e^{-k\cdot p_{\mathrm{EP}}}, \qquad k = 0.01. \tag{5.12}$$

5.5 Neoantigen Generation

The mutation system provides the molecular basis for adaptive immune recognition of tumor cells.

5.5.1 Neoantigen Definition

Neoantigens are short peptide fragments derived from mutated proteins. In the model, neoantigens are represented as 10-mer subsequences (10 consecutive amino acids) extracted from each translated protein:

$$\mathcal{N} = \bigcup_{g\in\mathcal{G}} \left\{ p_g^{\mathrm{mut}}[i : i{+}10] \;\middle|\; 0 \le i \le 9 \right\}, \tag{5.13}$$

where p_g^{mut} is the mutated protein sequence for gene g and the set union is taken over all 16 genes.

5.5.2 Self-Antigens

The set of self-antigens is computed from the wildtype protein sequences using the same 10-mer extraction:

$$\mathcal{S} = \bigcup_{g\in\mathcal{G}} \left\{ p_g^{\mathrm{wt}}[i : i{+}10] \;\middle|\; 0 \le i \le |p_g^{\mathrm{wt}}| - 10 \right\}. \tag{5.14}$$

Self-antigens are computed once (as a class-level cached set) since the wildtype sequences are identical across all cells.

5.5.3 TCR Matching

T cell receptors (TCRs) recognize neoantigens through a matching mechanism based on Hamming distance:

1. The 10-mer antigen peptide is hashed to an 8-bit value using SHA-256 (taking the first byte of the digest).
2. The T cell's receptor is a random 8-bit integer generated at initialization.
3. The Hamming distance (number of differing bits) between the antigen hash and the receptor is computed.

4. A match is declared if the antigen is *not* a self-antigen and the Hamming distance is within a threshold:

$$\text{match}(a, r) = \big(a \notin \mathcal{S}\big) \ \wedge \ \big(d_H\big(h(a),\, r\big) \leq 3 + \Delta_r\big), \tag{5.15}$$

where $h(a)$ is the 8-bit hash of antigen a, r is the receptor value, d_H denotes Hamming distance, and Δ_r is the `receptor_threshold_variation` parameter (default 1, giving a threshold of 4). This mechanism ensures that T cells only respond to foreign (non-self) peptides and provides a stochastic, receptor-specific recognition model.

Chapter 6

Treatment System

This chapter describes the pharmacological treatment layer of the model, which simulates Immune Checkpoint Inhibitors (ICI), Tyrosine Kinase Inhibitors (TKI), and their combination. The treatment system is activated after a configurable number of simulation steps and modulates tumor and immune cell behaviors through drug-specific mechanisms.

6.1 Drug Architecture

The treatment system follows a composition-based design in which individual drugs are combined into a treatment regimen.

6.1.1 Abstract Drug Base Class

All drugs inherit from the abstract `Drug` base class (located in `src/treatments/drug.py`), which defines a single interface method:

```
class Drug(ABC):
    def __init__(self, model):
        self.model = model

    def step(self, proportion=1.0):
        raise NotImplementedError
```

Listing 6.1: Abstract Drug base class.

The `proportion` parameter controls what fraction of the treatment's total capacity is allocated to this drug. Each drug receives a reference to the model, granting access to all agent populations and parameters.

6.1.2 Treatment Composition

The `Treatment` class (in `src/treatments/treatment.py`) holds a list of `Drug` instances and distributes the treatment capacity equally among them:

```
class Treatment:
    def __init__(self, drugs):
        self.drugs = drugs

    def step(self):
```

```
        if not self.drugs:
            return
        for drug in self.drugs:
            drug.step(proportion=1.0 / len(self.drugs))
```

Listing 6.2: Treatment composition with proportional averaging.

With a single drug, `proportion` = 1.0; with two drugs (e.g., ICI + TKI), each receives `proportion` = 0.5. This proportional averaging models the clinical reality that combination therapy involves dose adjustments.

6.1.3 Treatment Activation

Treatment does not begin at the start of the simulation. Instead, it is activated after a configurable number of steps specified by the `treatment_start` patient parameter (default: step 100). Before this time point, the model simulates unperturbed tumor–immune dynamics, establishing a baseline disease state. At each step from `treatment_start` onward, `Treatment.step()` is called:

$$\text{Treatment active at step } t \iff t \geq t_{\text{start}}. \tag{6.1}$$

The available treatment configurations, selected via the `treatment` patient parameter, are:

- `"None"` – no drugs (control),
- `"ICI"` – immune checkpoint inhibitor monotherapy,
- `"TKI"` – tyrosine kinase inhibitor monotherapy,
- `"ICI+TKI"` – combination therapy.

6.2 Immune Checkpoint Inhibitors (ICI)

6.2.1 Biological Basis

The programmed death-1 (PD-1) receptor is expressed on activated T cells and serves as a negative regulator of T cell function. Its ligand, PD-L1 (encoded by the CD274 gene), is frequently upregulated on tumor cells in renal cell carcinoma. When PD-L1 on a tumor cell engages PD-1 on a T cell, the T cell's cytotoxic activity is suppressed—a process known as immune checkpoint inhibition [11].

Anti-PD-1 and anti-PD-L1 monoclonal antibodies (collectively termed ICI) block this interaction, restoring T cell effector function. In clinical practice, ICI therapy (e.g., nivolumab, pembrolizumab) has shown significant survival benefits in RCC patients [12, 10].

6.2.2 Implementation

The `ICIDrug` class (in `src/treatments/ici.py`) implements three effects per treatment step:

1. PD-L1 Blockade on Tumor Cells. For each tumor cell, with probability equal to `proportion` × `w_ici_effectiveness`:

$$P(\text{ICI_effect} = \texttt{True}) = p \cdot w_{\text{ICI}}, \tag{6.2}$$

where p is the drug proportion and w_{ICI} is the ICI effectiveness weight (default 1.0). When `ICI_effect` is `True`, the tumor cell's `get_PD1_inhibition()` method returns `False`, effectively disabling the checkpoint pathway for that cell.

2. T Cell Activation Restoration. For each T cell (across all six T cell subtypes: CD8 cytotoxic, CD8 naive, CD4 naive, CD4 Th1, CD4 Th2, and Treg), with the same probability:

$$\texttt{t_activation_effect} \leftarrow 1.0. \tag{6.3}$$

This resets the T cell's activation state to full capacity, counteracting the progressive exhaustion that occurs when T cells encounter PD-L1$^+$ tumor cells without treatment.

3. Immune Infiltration Enhancement. For immune cell types Th1, Th2, and MastCell, with the same probability, the `immune_infiltration_factor` is incremented:

$$\texttt{immune_infiltration_factor} \leftarrow \texttt{immune_infiltration_factor} + 1. \tag{6.4}$$

This models the clinical observation that ICI therapy promotes immune cell infiltration into the tumor mass, as the restored immune microenvironment generates stronger chemotactic signals.

6.3 Tyrosine Kinase Inhibitors (TKI)

6.3.1 Biological Basis

Vascular endothelial growth factor (VEGF) is a key driver of angiogenesis—the formation of new blood vessels that supply the tumor with nutrients and oxygen. Renal cell carcinoma is particularly dependent on VEGF signaling due to frequent VHL gene inactivation, making anti-angiogenic therapy a cornerstone of RCC treatment.

Tyrosine kinase inhibitors (e.g., axitinib, sunitinib) block the intracellular signaling domains of VEGF receptors and other receptor tyrosine kinases, thereby inhibiting tumor-driven angiogenesis and directly reducing tumor cell proliferation [12].

6.3.2 Implementation

The `TKIDrug` class (in `src/treatments/tki.py`) implements four effects:

1. Tumor Growth Suppression. For each tumor cell, with probability `proportion` × `w_tki_effectiveness`, the tumor growth effect is reduced:

$$\texttt{tumour_growth_effect} \leftarrow \texttt{tumour_growth_effect} \times (1 - \varepsilon), \tag{6.5}$$

where $\varepsilon = p \cdot w_{\text{TKI}}$ is the effective TKI strength.

2. Angiogenesis Inhibition. With the same per-cell probability, the angiogenesis effect is reduced:

$$\texttt{angiogenesis_effect} \leftarrow \texttt{angiogenesis_effect} \times (1 - \varepsilon). \tag{6.6}$$

This directly suppresses the tumor cell's ability to recruit new blood vessels, reducing nutrient supply and constraining spatial expansion.

3. Neoantigen Release. With probability ε, the tumor cell's `TKI_effect` flag is set to `True`. When a tumor cell with `TKI_effect = True` subsequently undergoes apoptosis, it releases its neoantigens to nearby dendritic cells (within radius 3), facilitating antigen presentation and downstream T cell activation. This models the immunogenic cell death induced by TKI therapy.

4. Treg Differentiation Suppression. For each CD4 naive T cell, with probability ε:

$$\texttt{treg_differentiation_effect} \leftarrow \texttt{treg_differentiation_effect} \times (1 - \varepsilon). \tag{6.7}$$

TKI therapy reduces the differentiation of naive $CD4^+$ T cells into immunosuppressive regulatory T cells (Tregs), thereby shifting the immune balance towards anti-tumor activity.

6.4 Combination Therapy

6.4.1 ICI + TKI

When both ICI and TKI are administered simultaneously, the `Treatment` class creates both drug instances, each receiving `proportion` $= 0.5$:

$$p_{\text{ICI}} = p_{\text{TKI}} = \frac{1}{2}. \tag{6.8}$$

The effective strength of each drug is halved compared to monotherapy, reflecting the dose adjustments common in combination regimens.

6.4.2 Synergistic Mechanisms

The combination of ICI and TKI exploits complementary mechanisms that produce synergistic anti-tumor effects:

1. **Vascular normalization:** TKI reduces aberrant tumor vasculature, which paradoxically improves blood flow to the tumor center. This normalized vasculature allows more immune cells to infiltrate the tumor mass.
2. **Restored immune function:** ICI blocks the PD-1/PD-L1 checkpoint, restoring the cytotoxic capacity of the infiltrating T cells. Without ICI, even well-infiltrated T cells may be rendered anergic by PD-L1 engagement.
3. **Immunogenic cell death:** TKI-induced tumor cell death releases neoantigens, priming new adaptive immune responses. ICI ensures that the T cells activated by these neoantigens remain functional.

4. **Treg reduction:** TKI suppresses Treg differentiation, reducing the immunosuppressive component of the tumor microenvironment, while ICI restores the activity of effector T cells.

6.4.3 Clinical Evidence

The clinical rationale for ICI + TKI combination therapy in RCC is well established. The KEYNOTE-426 trial demonstrated that pembrolizumab (anti-PD-1) combined with axitinib (TKI) significantly improved overall survival and progression-free survival compared to sunitinib monotherapy in advanced RCC [12]. Similarly, the JAVELIN Renal 101 trial showed that avelumab (anti-PD-L1) plus axitinib improved progression-free survival over sunitinib [10]. The ARON-1 study further examined sex-specific response patterns in this combination setting [13].

The model's treatment system is designed to reproduce the qualitative dynamics observed in these trials: combination therapy produces stronger tumor regression than either monotherapy, and the relative benefit depends on the patient's immune profile and sex-specific parameters (see Chapter 7).

Chapter 7

Sex Hormone System

This chapter describes the sex hormone subsystem, which models the immunomodulatory effects of estrogen, testosterone, and progesterone on the tumor microenvironment. Sex-specific differences in hormone profiles produce distinct immune dynamics, contributing to the well-documented sex disparity in RCC incidence and treatment response.

7.1 Biological Basis

7.1.1 Sex Differences in Renal Cell Carcinoma

Renal cell carcinoma exhibits a pronounced sex disparity: males are affected approximately twice as often as females (roughly 2:1 incidence ratio). However, the relationship between sex and treatment outcome is more nuanced. A landmark meta-analysis by Conforti et al. [3] demonstrated that the efficacy of immune checkpoint inhibitors varies significantly between sexes across multiple cancer types, with males generally showing a larger benefit from anti-PD-1/PD-L1 therapy. The ARON-1 study [13] further investigated sex-specific responses to ICI + TKI combination therapy in RCC, finding differential immune activation patterns between male and female patients.

These clinical observations suggest that sex hormones modulate the immune response in ways that affect both tumor susceptibility and treatment efficacy.

7.1.2 Estrogen

Estrogen exerts complex, concentration-dependent effects on the immune system. At low to moderate levels, estrogen promotes Th1-type (pro-inflammatory) immune responses, enhances IFN-γ production, upregulates MHC expression, and stimulates T cell activation and costimulation via CD28. At high concentrations, however, estrogen shifts the balance towards Th2-type (anti-inflammatory) responses, increases IL-4 and IL-13 production, upregulates CTLA-4 and PD-1 inhibitory pathways, and suppresses T cell activation. This biphasic behavior reflects the known immunological dichotomy between low-dose immunostimulatory and high-dose immunosuppressive effects of estrogen.

7.1.3 Testosterone

Testosterone is generally immunosuppressive at elevated levels. High testosterone concentrations reduce Th1 proliferation, decrease IFN-γ production, and promote immune

checkpoint engagement (CTLA-4, PD-1). Conversely, low testosterone levels are associated with enhanced Th1 responses and T cell activation. Testosterone also directly influences $CD8^+$ T cell population dynamics (see Section 7.4).

7.1.4 Progesterone

Progesterone has immunomodulatory properties that depend on concentration. At low levels, its effects are minimal. At moderate levels, progesterone promotes T cell differentiation and mildly enhances checkpoint inhibition. At high concentrations, progesterone strongly suppresses Th1 proliferation, promotes Th2 differentiation, and upregulates inhibitory checkpoints (CTLA-4, PD-1), producing a net immunosuppressive effect.

7.2 Hormone Agent Model

Sex hormones are modeled as discrete agents that diffuse through the simulation grid, interact with nearby immune cells, and decay over time.

7.2.1 Agent Type and Inheritance

The `SexHormone` class inherits directly from `repast4py.core.Agent` (rather than from the `Cell` base class used by other agents), reflecting the fact that hormones are signaling molecules rather than cells. Sex hormones are registered as agent type ID 16 (`AgentType.SEX_HORMONE`).

7.2.2 Hormone Types

Three hormone types are modeled, defined as an enumeration (`SexHormoneType`):

- `ESTROGEN` – primary female sex hormone,
- `TESTOSTERONE` – primary male sex hormone,
- `PROGESTERONE` – steroid hormone involved in immune regulation.

7.2.3 Spawning Mechanism

Hormone agents are spawned from blood vessel positions at each simulation step via the model's `spawn_hormones()` method. For each hormone type, a single spawn attempt is made per step, with success probability depending on the patient's sex:

Table 7.1: Hormone spawn probabilities by patient sex.

Sex	Estrogen	Progesterone	Testosterone
Male	0.2	0.1	1.0
Female	0.9	0.8	0.2

When a hormone is spawned, it is placed at the position of a randomly selected blood vessel agent, simulating hormonal delivery via the bloodstream.

7.2.4 Hormone Diffusion

Once spawned, hormone agents perform a biased random walk through the grid. At each step, the agent selects one of two movement strategies with equal probability:

1. **Biased diffusion:** the agent evaluates its Moore neighborhood (radius 1) and selects a target position with probability biased towards greater distance from blood vessels. The selection probability for neighbor c is:

$$P(c) = (1 - \alpha) \cdot \frac{1}{|\mathcal{N}|} + \alpha \cdot \frac{d(c)}{d_{\max} \cdot |\mathcal{N}|}, \tag{7.1}$$

   where $\alpha = 0.25$ is the drift strength, $d(c)$ is the Manhattan distance from c to the nearest blood vessel, $d_{\max}$ is the maximum such distance among all neighbors, and $|\mathcal{N}|$ is the neighborhood size.
2. **Simple diffusion:** the agent selects a move from a predefined set of directions, biased towards the positive y and z directions (away from the vascular channel).

Hormones that exit the grid boundaries are removed from the simulation, providing a natural mechanism for hormone clearance at the domain edges. The combination of biased diffusion and boundary removal limits the effective lifespan of hormone agents to approximately 50 steps, depending on grid dimensions and diffusion trajectory.

7.3 Hormone Effects on T Cells

T cells perceive nearby sex hormones and modulate their behavior accordingly. This mechanism is implemented in the `TCell` base class, ensuring that all T cell subtypes (CD8 cytotoxic, CD8 naive, CD4 naive, CD4 Th1, CD4 Th2, Treg) are affected.

7.3.1 Hormone Perception

T cells detect sex hormones in their local neighborhood via the `perceive_sex_hormone()` method, which queries the spatial index using `grid_utils` neighbor queries. Each T cell maintains a dictionary of perceived hormone levels, keyed by hormone type. When a hormone agent is perceived:

1. The T cell's perceived level for that hormone type is incremented.
2. The hormone agent is removed from the simulation (consumed).
3. A maximum of 9 hormones of each type can be perceived per step (capped at `MAX_LEVEL`).

Perceived hormone levels decay by 10% per step, modeling the transient nature of hormonal signaling.

7.3.2 Stimulation Level

The `sex_hormone_stimulation_level()` method returns the current perceived level of a given hormone type, providing a scalar measure of hormonal influence that is used by downstream effect computations.

7.3.3 Hormonal Influence Effects

The `CellHormonalInfluenceEffect` enumeration (in `src/systems/hormonal_effect.py`) defines 17 distinct effect channels through which hormones modulate T cell behavior:

Table 7.2: Hormonal influence effect channels.

Effect	Description
KILL_RATE	Cytotoxic kill rate modulation.
PROLIFERATION	General proliferation rate.
APOPTOSIS	Programmed cell death susceptibility.
ACTIVATION	T cell activation state.
DIFFERENTIATION	Differentiation probability (e.g., naive to effector).
TH1_PROLIFERATION	Th1-specific proliferation.
TH2_PROLIFERATION	Th2-specific proliferation.
IL_4	Interleukin-4 signaling effect.
IL_6	Interleukin-6 signaling effect.
IL_13	Interleukin-13 signaling effect.
IFN_GAMMA	Interferon-gamma signaling effect.
TNF_ALPHA	Tumor necrosis factor alpha effect.
MHC_I_UPREGULATION	MHC class I expression modulation.
MHC_II_UPREGULATION	MHC class II expression modulation.
COSTIMULATION_CD28	CD28 costimulatory signal strength.
CTLA4_SUPPRESSION	CTLA-4 inhibitory checkpoint engagement.
PD1_INHIBITION	PD-1 inhibitory checkpoint engagement.

7.3.4 Concentration-Dependent Effect Mapping

Each hormone type exerts different effects depending on its normalized concentration level. The perceived hormone level is normalized to a $[0, 1]$ scale using a threshold of 9 (the maximum level):

$$\hat{h} = \frac{h - 1}{\text{MAX_LEVEL} - 1}, \tag{7.2}$$

where h is the raw perceived level. Three concentration regimes are defined:

- **Low:** $\hat{h} < 0.33$
- **Medium:** $0.33 \leq \hat{h} < 0.66$
- **High:** $\hat{h} \geq 0.66$

For each regime, a predefined effect vector specifies integer-valued influence scores (typically ranging from -3 to $+3$) for each effect channel. These scores are weighted by hormone-specific importance weights before being accumulated:

$$w_{\text{estrogen}} = 0.45, \qquad w_{\text{testosterone}} = 0.30, \qquad w_{\text{progesterone}} = 0.25. \tag{7.3}$$

The weighted effects are accumulated into the T cell's `accumulated_effects` dictionary and subsequently transferred to the cell's `experienced_effects` via the `update_effects()` method.

Example: Estrogen at High Concentration. When estrogen is at high concentration ($\hat{h} \geq 0.66$), the effect vector includes: TH1_PROLIFERATION $= -3$ (strong suppression), TH2_PROLIFERATION $= +3$ (strong promotion), IL_4 $= +3$, IL_13 $= +3$, IFN_GAMMA $= -3$, ACTIVATION $= -2$, CTLA4_SUPPRESSION $= +3$, PD1_INHIBITION $= +2$, KILL_RATE $= -2$, and PROLIFERATION $= -1$. After

weighting by $w_{\text{estrogen}} = 0.45$, these effects produce a net immunosuppressive shift consistent with the biological role of high-dose estrogen.

7.4 Sex Drift

In addition to the direct hormonal effects on T cell behavior, the model implements a population-level sex drift mechanism that adjusts the CD8$^+$ naive T cell population based on patient sex.

7.4.1 Biological Basis

Testosterone stimulates the production and maturation of CD8$^+$ T cells in the thymus. Male patients, with higher testosterone levels, maintain larger CD8$^+$ T cell populations, which contributes to stronger cytotoxic anti-tumor responses. Conversely, female patients tend to have relatively fewer CD8$^+$ T cells but may exhibit enhanced CD4$^+$ T helper and antibody-mediated responses.

7.4.2 Implementation

The sex drift is implemented in the `RCCModel` class via the `init_sex_drift()` and `apply_sex_drift()` methods. The drift operates as a gradual population adjustment over the first n simulation steps (default $n = 15$):

$$\delta = \frac{d_{\max}}{n}, \tag{7.4}$$

where $d_{\max}$ is the `w_max_drift` parameter (default 0.25), representing the maximum fraction of the CD8 naive T cell population affected over the entire drift period. The drift direction depends on sex:

$$\sigma = \begin{cases} +1 & \text{if sex} = \text{Male (positive drift, population increase)}, \\ -1 & \text{if sex} = \text{Female (negative drift, population decrease)}. \end{cases} \tag{7.5}$$

At each step during the drift period, the step factor is computed as:

$$f_{\text{step}} = 1 + \sigma \cdot \delta. \tag{7.6}$$

The population adjustment proceeds as follows:

- If $f_{\text{step}} > 1$ (male, population increase): each CD8 naive T cell is duplicated with probability $|f_{\text{step}} - 1|$.
- If $f_{\text{step}} < 1$ (female, population decrease): each CD8 naive T cell is removed with probability $|f_{\text{step}} - 1|$.

The drift terminates after n steps, at which point the CD8 naive T cell population has been adjusted by approximately $\pm d_{\max} \times 100\%$ relative to its initial size. For the default parameters ($d_{\max} = 0.25$, $n = 15$), this produces a $\sim$25% population increase in males and a $\sim$25% decrease in females over the first 15 simulation steps.

7.4.3 Interaction with Hormonal Effects

The sex drift and the direct hormonal effects operate through complementary mechanisms:

- The **sex drift** adjusts the *initial population size* of $CD8^+$ T cells, reflecting the baseline sex difference in T cell counts.
- The **hormonal effects** modulate the *functional behavior* (kill rates, proliferation, activation) of all T cell types throughout the simulation.

Together, these mechanisms produce sex-specific immune dynamics that are consistent with clinical observations: male patients start with more $CD8^+$ T cells and experience testosterone-driven immune modulation, while female patients have fewer $CD8^+$ T cells but benefit from estrogen-mediated immune activation at low concentrations. The net effect on treatment outcome depends on the interplay between these factors and the treatment modality applied [3, 13].

Chapter 8

Implementation

This chapter describes the software implementation of the RCC agent-based model, covering the selection of the Repast4Py framework, the overall architecture, the custom spatial index design, agent serialization conventions, the porting process from Mesa, the glucose field implementation, and the simulation entry point.

8.1 Framework Selection

The choice of simulation framework is fundamental to the scalability and maintainability of any agent-based model. After evaluating the two most prominent Python-based ABM frameworks—Mesa [8] and Repast4Py [2]—the latter was selected for the RCC tumor microenvironment model.

8.1.1 Rationale for Repast4Py

Repast4Py offers several features that are critical for the present model:

(i) **Native 3D grids.** Repast4Py's `SharedGrid` class supports arbitrary dimensionality out of the box. In Mesa, only 2D grids are provided natively; extending them to three dimensions requires custom subclassing of `MultiGrid` with non-trivial modifications to the neighbor-query logic.

(ii) **MPI parallelism via mpi4py.** Repast4Py is designed around the Message Passing Interface (MPI) through the `mpi4py` library [4]. Agents are managed within a `SharedContext` that can distribute them across MPI ranks. This architecture enables future scaling to large grids without architectural changes to the model code.

(iii) **SharedContext for distributed agents.** The `SharedContext` abstraction provides a unified agent container that handles ghost-agent synchronization across rank boundaries, which is essential for multi-rank parallel execution.

(iv) **Python ecosystem compatibility.** Repast4Py is a pure Python library (with `mpi4py` as its primary compiled dependency), and integrates seamlessly with NumPy, SciPy, Optuna, Pandas, and Matplotlib—all of which are used in the RCC model.

8.1.2 Comparison with Mesa

Table 8.1 summarizes the key differences between Mesa and Repast4Py that influenced the framework selection.

Table 8.1: Feature comparison between Mesa and Repast4Py for the RCC model.

Feature	**Mesa**	**Repast4Py**
Grid	`MultiGrid` (custom 3D)	`SharedGrid` (native 3D)
Parallelism	None (single process)	MPI via `mpi4py`
Scheduling	Mesa scheduler	`schedule.init_schedule_runner`
Agent base class	`mesa.Agent(model)`	`repast4py.core.Agent(id, type, rank)`
Border handling	Torus / Non-torus	Sticky / Periodic
Occupancy	Single / Multi	Single / Multiple

The most consequential difference for this project is the parallelism model. Mesa is inherently single-process: all agents live in a single Python interpreter, and there is no built-in mechanism for distributing computation across cores or nodes. Repast4Py, by contrast, is architected from the ground up around MPI, making it possible to scale the simulation to larger tissue volumes in the future without redesigning the agent management infrastructure.

8.2 Architecture

8.2.1 Project Structure

The project is organized into a modular directory tree, separating agent definitions, systems (spatial index, glucose field, DNA, effects), treatment logic, parameter management, the model driver, data collection, machine learning, visualization, and tests.

```
rcc_repast4py/
|-- run.py                        # Entry point
|-- config/
|   `-- default_params.yaml       # Default parameter file
|-- src/
|   |-- agents/
|   |   |-- agent_types.py        # AgentType IntEnum (0-17)
|   |   |-- cell.py               # Base Cell(ABC, Agent) class
|   |   |-- tumor_cell.py         # TumorCell with Warburg glucose
|   |   |-- cd8_cytotoxic_t_cell.py
|   |   |-- cd8_t_cell.py
|   |   |-- cd4_t_cell.py
|   |   |-- cd4_t_cell_h1.py
|   |   |-- cd4_t_cell_h2.py
|   |   |-- regulatory_t_cell.py
|   |   |-- dendritic_cell.py
|   |   |-- plasmacitoid_dc.py
|   |   |-- macrophage_m1.py
|   |   |-- macrophage_m2.py
|   |   |-- natural_killer.py
|   |   |-- mast_cell.py
|   |   |-- neutrophil.py
```

```
|    |    |-- adipocyte.py
|    |    |-- blood.py
|    |    |-- sex_hormone.py
|    |    |-- cytokine.py
|    |    `-- t_cell.py             # Shared T-cell base
|    |-- systems/
|    |    |-- grid_utils.py         # 3D neighbor queries
|    |    |-- glucose_field.py      # GlucoseField class
|    |    |-- dna.py                # 16-gene DNA mutation system
|    |    |-- effect.py             # Effect accumulator
|    |    `-- hormonal_effect.py    # Hormonal effect system
|    |-- treatments/
|    |    |-- drug.py               # Drug base class
|    |    |-- ici.py                # ICIDrug
|    |    |-- tki.py                # TKIDrug
|    |    `-- treatment.py          # Treatment container
|    |-- parameters/
|    |    |-- parameters.py         # Base Parameters dataclass
|    |    |-- model_parameters.py   # ModelParameters (4 params)
|    |    |-- patient_parameters.py # PatientParameters (18 params)
|    |    `-- weight_parameters.py  # WeightParameters (92 params)
|    |-- model/
|    |    |-- rcc_model.py          # RCCModel main driver
|    |    |-- cell_adder.py         # Agent initialization
|    |    |-- observer.py           # Kill/event counters
|    |    `-- measures_utils.py     # Volume/concentration helpers
|    |-- data/
|    |    `-- data_collector.py     # Data collection utilities
|    |-- learning/
|    |    |-- train_model.py        # Optuna optimization
|    |    |-- best_parameters.py    # Best parameter storage
|    |    `-- data/
|    |        `-- ARON.csv          # ARON-1 clinical dataset
|    `-- visualization/
|        |-- plot_results.py       # Population dynamics plots
|        `-- plot_glucose.py       # Glucose field visualization
`-- tests/
     |-- test_dna.py
     |-- test_effects.py
     |-- test_glucose_field.py
     `-- test_single_step.py
```

Listing 8.1: Project directory tree.

8.2.2 Plain Python Model Class

Unlike Mesa, which requires the model class to inherit from `mesa.Model` and use Mesa's scheduler abstraction, the Repast4Py implementation uses a plain Python class, `RCCModel`. This class does not inherit from any framework base class; instead, it manually manages:

- A `SharedContext` instance for agent registration with Repast4Py.
- A `SharedGrid` instance for spatial placement.
- A custom `spatial_index` dictionary for efficient neighbor queries.
- A `GlucoseField` instance for metabolic dynamics.

- Agent lifecycle methods: `add_agent()`, `remove_agent()`, `move_agent()`.
- A standard library `random.Random` instance (`self.rng`) seeded deterministically.

There is no framework-imposed scheduler. Instead, Repast4Py's `schedule.init_schedule_runner(` creates an event-driven runner that invokes `model.step()` at each tick. The `step()` method itself handles the ordering of sub-steps (effect collection, treatment application, glucose field update, agent stepping, deferred movement, and hormone spawning) explicitly, providing full control over the simulation loop.

8.3 Spatial Index Design

8.3.1 Problem

Repast4Py's `SharedGrid` provides basic spatial placement and movement of agents, but it does *not* expose the `get_neighbors()` or `get_neighborhood()` methods that Mesa's grids provide. This means there is no built-in mechanism to query which agents are adjacent to a given position—a fundamental operation in the RCC model, where every agent interaction (phagocytosis, antigen presentation, killing, effect propagation) depends on spatial proximity.

8.3.2 Solution

The model maintains a manual spatial index implemented as a `defaultdict(list)` that maps each `(x, y, z)` coordinate tuple to the list of agents currently occupying that position:

```
from collections import defaultdict

self.spatial_index = defaultdict(list)
```

Listing 8.2: Spatial index declaration in `RCCModel.__init__()`.

This index is updated on every agent lifecycle operation:

- `add_agent(agent)`: appends the agent to `spatial_index[agent.pos]`.
- `remove_agent(agent)`: removes the agent from `spatial_index[agent.pos]`.
- `move_agent(agent, new_pos)`: removes from the old position list and appends to the new one.

8.3.3 Grid Utilities Module

The `src/systems/grid_utils.py` module provides five functions that operate on the spatial index:

1. `get_neighborhood_3d(width, height, depth, pos, radius, moore)`: Returns all coordinate tuples in the Moore (Chebyshev) or Von Neumann (Manhattan) neighborhood of a given position, excluding the center. For Moore neighborhoods at radius 1 in 3D, this yields up to 26 neighbors ($3^3 - 1$).

2. `get_agents_at_3d(spatial_index, pos)`: Returns the list of agents at a specific coordinate.

3. `get_neighbors_3d(spatial_index, grid_dims, pos, radius, moore)`: Returns all agents in the neighborhood of a position by iterating over the neighborhood coordinates and collecting agents from the spatial index.

4. `is_cell_empty_3d(spatial_index, pos)`: Returns `True` if no agents occupy the given position.

5. `get_empty_neighbors_3d(spatial_index, grid_dims, pos, radius, moore)`: Returns the list of neighboring positions that have no agents, used for cell duplication and random movement.

```python
def get_neighborhood_3d(width, height, depth, pos, radius=1, moore=True):
    x, y, z = pos
    neighborhood = []
    for dx in range(-radius, radius + 1):
        for dy in range(-radius, radius + 1):
            for dz in range(-radius, radius + 1):
                if dx == 0 and dy == 0 and dz == 0:
                    continue
                if not moore and abs(dx) + abs(dy) + abs(dz) > radius:
                    continue
                nx, ny, nz = x + dx, y + dy, z + dz
                if 0 <= nx < width and 0 <= ny < height and 0 <= nz < depth:
                    neighborhood.append((nx, ny, nz))
    return neighborhood
```

Listing 8.3: Moore neighborhood computation in 3D.

The boundary check (`0 <= nx < width`, etc.) ensures that the neighborhood is clipped at grid edges, consistent with the sticky border semantics of the `SharedGrid`.

8.4 Agent Serialization

8.4.1 Agent Identity in Repast4Py

Repast4Py requires every agent to be constructed with three arguments: an integer `id`, an integer `type`, and an integer `rank` (the MPI rank that owns the agent). The framework stores these as a `uid` tuple `(id, type, rank)` on every agent. This design supports distributed serialization, as agents can be uniquely identified and routed across MPI ranks.

8.4.2 Agent Type Registry

The model defines an `AgentType IntEnum` that maps all 18 agent types to integer IDs in the range $[0, 17]$:

```python
from enum import IntEnum

class AgentType(IntEnum):
    TUMOR_CELL = 0
    CD8_CYTOTOXIC_T_CELL = 1
    CD8_NAIVE_T_CELL = 2
    CD4_NAIVE_T_CELL = 3
    CD4_HELPER1_T_CELL = 4
    CD4_HELPER2_T_CELL = 5
    REGULATORY_T_CELL = 6
    DENDRITIC_CELL = 7
    PLASMACITOID_DC = 8
    MACROPHAGE_M1 = 9
    MACROPHAGE_M2 = 10
    NATURAL_KILLER = 11
    MAST_CELL = 12
    NEUTROPHIL = 13
    ADIPOCYTE = 14
    BLOOD = 15
    SEX_HORMONE = 16
    CYTOKINE = 17
```

Listing 8.4: Agent type registry (`src/agents/agent_types.py`).

8.4.3 The `save()` Method

Each agent implements a `save()` method that returns a tuple of `(uid, state_tuple)` for distributed serialization. The base implementation in `Cell` is:

```python
def save(self):
    return (self.uid, (self.pos, self._alive))
```

Listing 8.5: Agent serialization via `save()`.

Subclasses extend the state tuple to include type-specific fields (e.g., the `TumorCell` includes its `DNA` state, blood access flag, and experienced effects).

8.4.4 Type Identification

Because Repast4Py agents carry their type as the second element of the `uid` tuple, type identification uses integer comparison rather than Python's `isinstance()` check:

```python
# Instead of isinstance(agent, TumorCell):
if agent.uid[1] == AgentType.TUMOR_CELL:
    # handle tumor cell
```

Listing 8.6: Type identification pattern.

This approach is both faster (integer comparison vs. method resolution order traversal) and compatible with ghost agents that may not carry the full Python class hierarchy across MPI ranks.

8.5 Mesa to Repast4Py Porting

The original RCC model was implemented in Mesa [8, 6]. Porting it to Repast4Py required systematic translation of Mesa idioms to their Repast4Py equivalents. Table 8.2 summarizes the key mappings.

Table 8.2: Key API mapping from Mesa to Repast4Py.

Mesa	**Repast4Py**
`mesa.Agent(model)`	`repast4py.core.Agent(id, type, rank)`
`self.model.random`	`model.rng` (stdlib `random.Random`)
`self.model.grid.get_neighbors()`	`grid_utils.get_neighbors_3d(spatial_index, ...)`
`isinstance(agent, TumorCell)`	`agent.uid[1] == AgentType.TUMOR_CELL`
`self.model.schedule.agents`	`model._all_agents`
`model.agents_by_type[Type]`	`model.get_agents_by_type_id(type_id)`
`TumorCell.blood_source` (class var)	`model.tumor_blood_sources`
`Cell.search_dimension` (class var)	`model.cell_search_dimension`

8.5.1 Random Number Generation

Mesa provides a model-level `self.random` attribute backed by a NumPy random generator. In Repast4Py, no such attribute is provided, so the model creates a standard library `random.Random` instance seeded with `random_seed + rank`:

```
import random
self.rng = random.Random(self.model_params.random_seed + self.rank)
```

Listing 8.7: RNG initialization in RCCModel.

All agent behaviors reference `self.model.rng` for stochastic decisions, ensuring reproducibility at a given seed.

8.5.2 Class-Level State to Model-Level State

In the Mesa version, certain state was stored as class variables (e.g., `TumorCell.blood_source` tracked all tumor cells with blood vessel access). In the Repast4Py version, this state is moved to the model instance to avoid issues with class-level state in a distributed setting:

```
# In RCCModel.__init__():
self.tumor_blood_sources = []
self.cell_search_dimension = 5
```

Listing 8.8: Class-level state moved to model.

8.5.3 Deferred Movement

The deferred movement pattern from the Mesa version is preserved. During their `step()` method, agents set a `_desired_pos` attribute rather than moving immediately. After all agents have been stepped, the model calls `move_all_agents()`, which resolves all pending movements in a single pass:

```
def move_all_agents(self):
    for agent in list(self._all_agents):
        if (getattr(agent, '_alive', True)
                and isinstance(agent, Cell)
                and hasattr(agent, '_desired_pos')
                and agent._desired_pos is not None
                and agent._desired_pos != agent.pos):
            self.move_agent(agent, agent._desired_pos)
```

Listing 8.9: Deferred movement resolution.

This approach prevents order-of-execution artifacts where early-stepping agents could influence the spatial configuration seen by later-stepping agents within the same tick.

8.6 Glucose Field Implementation

The glucose concentration field is implemented as a standalone class, `GlucoseField`, in `src/systems/glucose_field.py`. It uses pure NumPy arrays and SciPy convolution rather than Repast4Py's built-in value layers, for two reasons: (i) full control over the diffusion kernel and boundary conditions, and (ii) performance, since NumPy operations are vectorized at the C level.

8.6.1 Data Structure

The field is stored as a NumPy 3D array of shape `(width, height, depth)` with `float64` precision:

```
self.field = np.full((width, height, depth), initial_concentration,
                     dtype=np.float64)
```

Listing 8.10: Glucose field initialization.

8.6.2 Diffusion Kernel

Diffusion is computed via the 3D discrete Laplacian, applied using `scipy.ndimage.convolve`. The kernel is a $3 \times 3 \times 3$ array with:

- Center value: -6
- Six face-adjacent neighbors (along $\pm x$, $\pm y$, $\pm z$): $+1$ each
- All other entries: 0 (edge and corner neighbors are not included in the 6-connected Laplacian)

```
self._kernel = np.zeros((3, 3, 3), dtype=np.float64)
self._kernel[1, 1, 0] = 1.0    # z-1
self._kernel[1, 1, 2] = 1.0    # z+1
self._kernel[1, 0, 1] = 1.0    # y-1
self._kernel[1, 2, 1] = 1.0    # y+1
self._kernel[0, 1, 1] = 1.0    # x-1
self._kernel[2, 1, 1] = 1.0    # x+1
self._kernel[1, 1, 1] = -6.0   # center
```

Listing 8.11: Laplacian kernel construction.

8.6.3 Boundary Conditions

Zero-flux (Neumann) boundary conditions are enforced by passing `mode='nearest'` to the convolution function. This causes boundary voxels to replicate their own values when the kernel extends beyond the grid, effectively setting the normal derivative of the concentration to zero at all boundaries. The gradient computation uses clamped central finite differences with the same Neumann semantics.

8.6.4 Separation from Repast4Py Value Layers

Repast4Py provides its own `SharedValueLayer` for continuous fields. However, the glucose field implementation deliberately avoids this abstraction for several reasons:

1. The `SharedValueLayer` is designed for distributed access patterns that introduce synchronization overhead unnecessary for single-rank execution.
2. Direct NumPy array access allows use of SciPy's optimized convolution routines without intermediate copying.
3. The pure-NumPy approach simplifies debugging and testing, as the field can be inspected and manipulated with standard array operations.

8.7 Entry Point and Execution

The simulation is launched from `run.py`, which follows the standard Repast4Py execution pattern:

```
from mpi4py import MPI
from repast4py import schedule
from src.model.rcc_model import RCCModel

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()

    model = RCCModel(comm=comm)

    runner = schedule.init_schedule_runner(comm)
    runner.schedule_repeating_event(1, 1, model.step)
```

```
    runner.schedule_stop(model.max_steps)
    runner.schedule_end_event(model.log_data)

    runner.execute()

if __name__ == '__main__':
    main()
```

Listing 8.12: Simulation entry point (`run.py`).

8.7.1 Execution Command

The simulation is invoked via MPI:

```
mpirun -n 1 python run.py
```

The `-n 1` flag specifies a single MPI rank, which is the current target configuration. The architecture supports multi-rank execution by increasing the rank count; in that scenario, the `SharedContext` would distribute agents across ranks and the `SharedGrid` would handle ghost-agent synchronization via buffer regions.

8.7.2 Simulation Loop

The Repast4Py schedule runner calls `model.step()` at every tick (starting at tick 1, repeating every 1 tick). Within `step()`, the following sub-steps are executed in order:

1. **Data collection**: Record current population counts and glucose field statistics.
2. **Termination check**: Evaluate whether the simulation should stop (tumor eliminated, maximum tumor count reached, growth threshold exceeded, or maximum steps reached).
3. **Effect propagation**: Cells collect and apply effects from their neighbors.
4. **Treatment**: If the current step exceeds the treatment start time, apply ICI and/or TKI drug effects.
5. **Sex drift**: Apply sex-specific CD8+ T cell population adjustment.
6. **Glucose field step**: Inject glucose from blood vessels, diffuse, and decay.
7. **Blood management**: Angiogenesis and blood access refresh for tumor cells.
8. **Search dimension update**: Adjust the agent search radius based on tumor population size.
9. **Agent stepping**: Shuffle all agents and execute their individual `step()` methods.
10. **Deferred movement**: Resolve all pending agent movements.
11. **Hormone spawning**: Spawn sex hormones from blood vessels.
12. **Step counter increment**.

When the simulation terminates (either by the schedule runner reaching `max_steps` or by the model setting `self.running = False`), the end event triggers `model.log_data()`, which writes the collected time-series data to `logs/simulation_log.csv`.

Chapter 9

Parameters and Optimization

This chapter documents the complete parameter space of the RCC model, organized into three categories (model, patient, and weight parameters), with detailed attention to the 8 newly introduced glucose parameters. It then describes the Bayesian optimization pipeline used to calibrate the model against clinical data.

9.1 Parameter Categories

The model's parameter space is partitioned into three `dataclass`-based parameter objects, each inheriting from a common `Parameters` base class that provides serialization, XML loading, and keyword-argument initialization.

9.1.1 ModelParameters

`ModelParameters` controls the simulation geometry and execution. It contains 4 parameters:

Table 9.1: Model parameters.

Parameter	Default	Description
`volume`	0.0001	Simulated tissue volume in mL. Determines the 3D grid dimension via `get_dimension_from_volume()`.
`block_size`	10	Spatial resolution in μm per grid cell.
`random_seed`	1	Seed for the `random.Random` instance; added to MPI rank for per-rank determinism.
`max_steps`	500	Maximum number of simulation ticks before termination.

9.1.2 PatientParameters

`PatientParameters` encodes patient-specific clinical and biological features. It contains 18 parameters:

Table 9.2: Patient parameters.

Parameter	Default	Description
`sex`	`'F'`	Patient sex (`'F'` or `'M'`). Controls hormone chances, CD8+ drift direction, and initial DNA mutations.
`BMI`	22.0	Body mass index (kg/m^2). Modulates Treg differentiation, macrophage mutation, and NK cell kill rate via BMI weight parameters.
`treatment`	`'ICI+TKI'`	Treatment modality: `'None'`, `'ICI'`, `'TKI'`, or `'ICI+TKI'`.
`treatment_start`	100	Simulation step at which treatment begins.
Immune cell concentrations (cells/mL):		
`ctc_concentration`	80 000	Cytotoxic T cell (CD8+) concentration.
`neutrophil_concentration`	40 000	Neutrophil concentration.
`mast_cell_concentration`	40 000	Mast cell concentration.
`treg_concentration`	40 000	Regulatory T cell concentration.
`pdc_concentration`	40 000	Plasmacytoid dendritic cell concentration.
`th1_concentration`	40 000	CD4+ Th1 helper cell concentration.
`th2_concentration`	40 000	CD4+ Th2 helper cell concentration.
`dc_concentration`	80 000	Dendritic cell concentration.
`m1_concentration`	40 000	M1 macrophage concentration.
`m2_concentration`	40 000	M2 macrophage concentration.
`nkl_concentration`	160 000	Natural killer cell concentration.
`cd4_concentration`	80 000	Naïve CD4+ T cell concentration.
`cd8_concentration`	80 000	Naïve CD8+ T cell concentration.
`NKL_delta`	16 000	NK cell count difference (used in sex-specific adjustments).

9.1.3 WeightParameters

`WeightParameters` is the largest parameter class, containing all behavioral weights, biases, and system-level tuning knobs. In total, it comprises 84 original parameters plus 8 new glucose parameters, for a total of 92 tunable values. These are detailed in Sections 9.2 and 9.3.

9.2 Weight Parameters

Weight parameters control the probability thresholds, scaling factors, and biases that govern agent behaviors. They are organized by the agent type or subsystem they influence.

Bias parameters (prefixed with `b_`) are additive offsets to the corresponding weight's contribution, allowing the optimization to shift decision boundaries.

9.2.1 BMI Weights (4 parameters)

These weights modulate the influence of body mass index on immune cell behavior:

Table 9.3: BMI weight parameters.

Parameter	Default	Description
`w_BMI_on_treg_diff`	0.01	BMI influence on Treg differentiation probability.
`w_BMI_on_m1_mutation`	0.01	BMI influence on M1 macrophage mutation threshold.
`w_BMI_on_m2_mutation`	0.01	BMI influence on M2 macrophage mutation threshold.
`w_BMI_nkl_kill_rate`	0.01	BMI influence on NK cell kill rate.

9.2.2 Macrophage M1 Weights (6 weights + 2 biases)

Table 9.4: Macrophage M1 weight parameters.

Parameter	Default	Description
`w_m1_mutation`	1.0	M1 macrophage mutation threshold scaling.
`b_m1_mutation`	0.0	Bias for M1 mutation threshold.
`w_m1_move`	1.0	M1 macrophage look-up movement radius scaling.
`w_m1_phagocytosis`	1.0	M1 phagocytosis chance scaling.
`w_m1_digest`	1.0	M1 digestion rate scaling.
`w_m1_t_kill_rate`	1.0	M1 effect on T cell kill rate.
`w_m1_th1_proliferation`	1.0	M1 effect on Th1 proliferation.
`w_th1_proliferation`	1.0	Th1 proliferation scaling (via M1 influence).
`b_th1_proliferation`	0.0	Bias for Th1 proliferation.

9.2.3 Macrophage M2 Weights (5 weights + 1 bias)

Table 9.5: Macrophage M2 weight parameters.

Parameter	Default	Description
`w_m2_mutation`	1.0	M2 macrophage mutation threshold scaling.
`b_m2_mutation`	0.0	Bias for M2 mutation threshold.
`w_m2_move`	1.0	M2 macrophage look-up movement radius.
`w_m2_t_kill_rate`	1.0	M2 effect on T cell kill rate.
`w_m2_tumour_growth`	1.0	M2 effect on tumour growth promotion.
`w_m2_angiogenesis`	1.0	M2 effect on angiogenesis.

9.2.4 Tumor Cell Weights (8 weights + 3 biases)

Table 9.6: Tumor cell weight parameters.

Parameter	Default	Description
`w_tumor_apoptosis_eff`	1.0	Effect-driven apoptosis probability scaling.
`w_tumor_apoptosis_dna`	1.0	DNA-driven extra suppression chance scaling.
`b_tumor_apoptosis`	0.0	Bias for apoptosis probability.
`w_tumor_growth_eff`	0.6	Effect-driven growth probability scaling.
`w_tumor_growth_dna`	1.0	DNA-driven extra proliferation chance scaling.
`b_tumor_growth`	0.0	Bias for growth probability.
`w_tumor_angiogenesis`	1.0	Angiogenesis initiation probability scaling.
`b_tumor_angiogenesis`	0.0	Bias for angiogenesis probability.
`w_antigen_presentation`	1.0	Antigen presentation chance scaling.
`b_antigen_presentation`	0.0	Bias for antigen presentation.
`w_angiogenesis_tumor_growth`	1.0	Angiogenesis contribution to growth (when blood access is present).
`w_gene_pd1_inhibition`	1.0	PD-1/PD-L1 checkpoint pathway inhibition scaling.

9.2.5 CD4+ T Cell Weights (9 weights + 3 biases)

Table 9.7: CD4+ T cell weight parameters.

Parameter	Default	Description
w_cd4_treg_diff_effect	1.0	Effect-driven Treg differentiation scaling.
w_cd4_treg_diff_horm	1.0	Hormone-driven Treg differentiation scaling.
b_cd4_treg_diff	0.0	Bias for Treg differentiation.
w_cd4_th1_proliferation_effect	1.0	Effect-driven Th1 proliferation scaling.
w_cd4_th1_proliferation_horm	1.0	Hormone-driven Th1 proliferation scaling.
b_cd4_th1_proliferation	0.0	Bias for Th1 proliferation.
w_cd4_th1_spawn_m1	1.0	Th1 spawning of M1 macrophages.
w_cd4_th1_spawn_dc	1.0	Th1 spawning of dendritic cells.
w_cd4_th2_spawn_m1	1.0	Th2 spawning of M1 macrophages.
w_cd4_th2_spawn_t	1.0	Th2 spawning of T cells.
w_cd4_th2_proliferation_effect	1.0	Effect-driven Th2 proliferation scaling.
w_cd4_th2_proliferation_horm	1.0	Hormone-driven Th2 proliferation scaling.
b_cd4_th2_proliferation	0.0	Bias for Th2 proliferation.

9.2.6 Cytotoxic T Cell Weights (5 weights + 2 biases)

Table 9.8: Cytotoxic T cell (CD8+) weight parameters.

Parameter	Default	Description
w_cytotoxic_proliferation	1.0	Proliferation probability scaling.
w_cytotoxic_apoptosis	1.0	Apoptosis probability scaling.
b_cytotoxic_apoptosis	0.0	Bias for cytotoxic T cell apoptosis.
w_cytotoxic_kill	1.0	Tumor cell kill probability scaling.
b_cytotoxic_kill	0.0	Bias for cytotoxic kill probability.
w_cytotoxic_pd1_inhibition	1.0	PD-1 inhibition susceptibility scaling.
w_cytotoxic_move	1.0	Movement radius scaling for cytotoxic T cells.

9.2.7 Mast Cell Weights (6 parameters)

Table 9.9: Mast cell weight parameters.

Parameter	Default	Description
`w_mast_cell_angiogenesis`	1.0	Mast cell contribution to angiogenesis effect.
`w_mast_cell_m1_mutation`	1.0	Mast cell effect on M1 mutation.
`w_mast_cell_t_kill_rate`	1.0	Mast cell effect on T cell kill rate.
`w_mast_cell_tumour_apoptosis`	1.0	Mast cell contribution to tumour apoptosis.
`w_mast_cell_tumour_growth`	1.0	Mast cell contribution to tumour growth.
`w_mast_cell_spawn_dc`	1.0	Mast cell probability of spawning dendritic cells.

9.2.8 NK Cell Weights (3 weights + 1 bias)

Table 9.10: Natural killer cell weight parameters.

Parameter	Default	Description
`w_natural_killer_kill_rate`	1.0	NK cell direct kill rate scaling.
`b_natural_killer_kill_rate`	0.0	Bias for NK cell kill rate.
`w_nkl_t_kill_rate`	1.0	NK cell effect on T cell kill rate amplification.

9.2.9 Plasmacytoid DC Weights (6 weights + 1 bias)

Table 9.11: Plasmacytoid dendritic cell weight parameters.

Parameter	Default	Description
`w_pdc_nkl_spawn`	1.0	pDC probability of spawning NK cells.
`b_pdc_nkl_spawn`	0.0	Bias for pDC NK cell spawning.
`w_pdc_angiogenesis`	1.0	pDC contribution to angiogenesis effect.
`w_pdc_treg_diff`	1.0	pDC effect on Treg differentiation.
`w_pdc_t_proliferation`	1.0	pDC effect on T cell proliferation.
`w_pdc_t_kill`	1.0	pDC effect on T cell kill rate.
`w_pdc_nkl_kill`	1.0	pDC effect on NK cell kill rate.

9.2.10 Treg Weights (5 parameters)

Table 9.12: Regulatory T cell weight parameters.

Parameter	Default	Description
`w_treg_t_kill_rate`	1.0	Treg suppression of T cell kill rate.
`w_treg_t_proliferation`	1.0	Treg suppression of T cell proliferation.
`w_treg_t_apoptosis`	1.0	Treg induction of T cell apoptosis.
`w_treg_activation`	1.0	Treg activation threshold scaling.
`w_treg_dc_phagocytosis`	1.0	Treg effect on DC phagocytosis.

9.2.11 Global Weights (4+ parameters)

Table 9.13: Global weight parameters.

Parameter	Default	Description
`w_search_dimension`	0.5	Controls adaptive search radius for agent interactions: $\text{dim} = \max(5, \lfloor n_{\text{tumor}}/(10 \cdot w) \rfloor)$.
`w_tumour_growth_threshold`	1.0	Terminal condition threshold for mean tumour growth rate.
`w_cell_base_death_prob`	0.009	Per-step base death probability for all cells.
`w_progressive_exhaustion`	0.05	Rate of progressive immune cell exhaustion.
`w_ici_effectiveness`	1.0	ICI drug effectiveness scaling.
`w_tki_effectiveness`	1.0	TKI drug effectiveness scaling.
`w_sex_hormone_cd8`	1.0	Sex hormone influence on CD8+ population.
`w_dc_phagocytosis_effect`	1.0	DC phagocytosis effect scaling.
`b_dc_phagocytosis`	0.0	Bias for DC phagocytosis.
`w_max_drift`	0.25	Maximum sex-specific CD8+ drift over the drift period.
`receptor_threshold_variation`	1	Integer variation in TCR matching threshold.

9.3 Glucose Parameters

The glucose microenvironment system introduces 8 new weight parameters, all prefixed with `w_glucose_`. These parameters control the physical dynamics of glucose diffusion,

the metabolic consumption rates, and the coupling between glucose availability and agent behavior.

Table 9.14: Glucose weight parameters.

Parameter	Default	Description
`w_glucose_diffusion`	0.1	Diffusion coefficient D for the 3D Laplacian update. Stability requires $D < 1/6 \approx 0.167$; the default value of 0.1 provides moderate diffusion well within the stable regime.
`w_glucose_decay`	0.01	Natural glucose decay rate per step. Applied multiplicatively as $(1 - d)$ to the entire field, modeling background metabolic consumption by tissue not explicitly represented as agents.
`w_glucose_source_rate`	5.0	Amount of glucose injected per blood vessel agent per step. Blood vessels act as point sources, creating localized glucose-rich zones that subsequently diffuse outward.
`w_glucose_tumor_consumption`	2.0	Glucose consumed per tumor cell per step. Reflects the elevated glycolytic rate characteristic of the Warburg effect [14]. Tumor cells consume approximately 4× the glucose of immune cells.
`w_glucose_immune_consumption`	0.5	Glucose consumed per immune cell per step. Represents the metabolic cost of immune cell activation and effector function.
`w_glucose_growth_sensitivity`	1.0	Sensitivity of tumor growth probability to local glucose concentration. The glucose factor is computed as $\min(1, C \cdot s)$, where C is the local concentration and s is this sensitivity parameter.
`w_glucose_immune_sensitivity`	1.0	Sensitivity of immune cell effector function to local glucose concentration. Controls how strongly glucose deprivation impairs immune cell activity.
`w_glucose_chemotaxis_strength`	0.3	Probability that an immune cell follows the glucose gradient (chemotaxis) rather than performing a random walk. At each step, with probability $p = 0.3$, the cell moves in the direction of increasing glucose concentration.

9.3.1 Parameter Interactions

The glucose parameters interact with the existing weight parameters in several ways:

- **Tumor growth modulation**: The tumor cell duplication probability (governed by `w_tumor_growth_eff` and `w_tumor_growth_dna`) is multiplied by a glucose factor $\min(1, C \cdot \texttt{w_glucose_growth_sensitivity})$. When glucose is abundant, this factor approaches 1 and growth proceeds at the base rate; when glucose is depleted, growth is suppressed.
- **Immune function modulation**: Immune cell kill rates and activation probabilities can be scaled by an analogous glucose factor using `w_glucose_immune_sensitivity`, coupling metabolic resource availability to immune effectiveness.
- **Spatial competition**: The balance between `w_glucose_source_rate` (supply from blood vessels) and `w_glucose_tumor_consumption` (demand from tumor cells) determines the spatial glucose profile. High tumor consumption relative to supply creates depletion zones that impair both further tumor growth and immune infiltration.

9.4 Bayesian Optimization

9.4.1 Framework

The model uses Optuna [1] for Bayesian hyperparameter optimization. Optuna's Tree-structured Parzen Estimator (TPE) sampler efficiently explores high-dimensional parameter spaces by building probabilistic models of the objective function landscape.

9.4.2 Objective Function

The optimization objective is to minimize the prediction error of the model against the ARON-1 clinical dataset. For each Optuna trial, the following process is executed:

1. A set of weight parameters is suggested by the TPE sampler within predefined bounds.
2. For each patient case in the ARON-1 dataset, a simulation is run with the patient's clinical features (sex, BMI, treatment, immune cell concentrations) and the suggested weight parameters.
3. The simulation outcome (survival/death, number of steps) is compared against the clinical outcome.
4. The per-case error is computed as:

$$\text{error}_i = \begin{cases} (\text{steps}_i - \text{OS}_i)^2 & \text{if prediction matches outcome} \\ \text{HIGH_PENALTY}^2 & \text{if prediction contradicts outcome} \end{cases} \tag{9.1}$$

   where OS_i is the observed overall survival (in steps) and $\text{HIGH_PENALTY} = 500$.
5. The trial objective value is the mean error across all cases.

9.4.3 Search Space

The search space is defined in the `suggest_parameters()` function and covers the full 92-parameter weight space:

- **Positive weights** (62 parameters): Sampled log-uniformly in $[0.25, 4.0]$. The log-uniform distribution ensures adequate exploration of both small and large scaling factors.
- **Biases** (15 parameters): Sampled uniformly in $[-1.0, 1.0]$.
- **Effectiveness weights**: `w_ici_effectiveness` and `w_tki_effectiveness` in $[0.0, 1.0]$.
- **Threshold weights**: `w_tumour_growth_threshold` in $[0.75, 0.99]$; `w_cell_base_death_prob` in $[0.001, 0.1]$; `w_progressive_exhaustion` in $[0.005, 0.3]$.
- **Integer parameters**: `receptor_threshold_variation` in $\{0, 1, 2, 3\}$.
- **Glucose parameters** (8 parameters):
  - `w_glucose_diffusion` in $[0.01, 0.16]$ (respecting the $D < 1/6$ stability constraint).
  - `w_glucose_decay` in $[0.001, 0.1]$.
  - `w_glucose_source_rate` in $[1.0, 20.0]$.
  - `w_glucose_tumor_consumption` in $[0.5, 10.0]$.
  - `w_glucose_immune_consumption` in $[0.1, 2.0]$.
  - `w_glucose_growth_sensitivity` in $[0.1, 5.0]$.
  - `w_glucose_immune_sensitivity` in $[0.1, 5.0]$.
  - `w_glucose_chemotaxis_strength` in $[0.05, 0.8]$.

9.4.4 Optimization Configuration

The optimization runs for $N = 250$ trials, each evaluating every patient case in the ARON-1 dataset. Training uses `MPI.COMM_SELF` to create isolated single-rank model instances, avoiding MPI communication overhead during optimization. Trial errors and the best-so-far error are logged to `data/trial_errors.csv` for convergence analysis.

```
study = optuna.create_study(direction="minimize")
study.optimize(wrapped_objective, n_trials=250)
```

Listing 9.1: Optuna study creation and execution.

9.5 Clinical Dataset

9.5.1 ARON-1 Study

The ARON-1 study is a clinical trial of advanced renal cell carcinoma patients treated with immune checkpoint inhibitors (ICI), tyrosine kinase inhibitors (TKI), or their combination [12, 10]. The dataset provides the ground truth against which the model is calibrated.

9.5.2 Dataset Structure

The clinical dataset is stored in `src/learning/data/ARON.csv` and contains the following features for each patient case:

- **Patient features**: sex (`'F'`/`'M'`), BMI, treatment type (ICI, TKI, or ICI+TKI).
- **Immune cell concentrations**: 13 cell type concentrations matching the `PatientParameters` fields.
- **Outcome**: Overall survival (OS) in time units, and a binary death indicator.

The dataset is loaded with Pandas, with the treatment column parsed as a list of strings to support combination therapies:

```
def load_dataset(file_path: str) -> pd.DataFrame:
    def treatment_to_list(treatment_str):
        return [t.strip() for t in treatment_str.split('+')]
    return pd.read_csv(file_path,
                       converters={"treatment": treatment_to_list})
```

Listing 9.2: Dataset loading.

Each row of the dataset is used to parameterize a simulation run during optimization, with the patient's clinical features passed as keyword arguments to `RCCModel`. The simulation's survival prediction is then compared against the clinical outcome to compute the trial error.

Chapter 10

Simulation Results

This chapter describes the expected simulation outcomes for the RCC agent-based model across different scenarios: baseline (no treatment), treatment modalities, sex-specific responses, glucose field dynamics, and validation against clinical data. Since the model is designed for systematic calibration via Bayesian optimization (Chapter 9), the results presented here describe the qualitative behavior and expected trends rather than specific numerical outputs, which depend on the optimized parameter configuration.

10.1 Baseline Simulation

The baseline scenario runs the simulation with default parameters and no treatment (`treatment='None'`). This configuration serves as a control against which treatment effects are measured.

10.1.1 Expected Tumor Dynamics

In the absence of treatment, the model is expected to produce unbounded tumor growth. The initial tumor mass of approximately 10 cells proliferates driven by the combined effects of the tumor growth rate, blood vessel access via angiogenesis, and DNA-encoded proliferation advantages. The growth dynamics follow an approximately sigmoid curve:

1. **Early phase (steps 0–50)**: Slow growth as the small tumor mass has limited blood vessel access and the immune system is still active. Glucose is abundant throughout the grid.

2. **Exponential phase (steps 50–200)**: Angiogenesis establishes blood supply to the tumor periphery, accelerating growth. Tumor cells with favorable DNA mutations (oncogene activation, tumor suppressor loss) dominate the population.

3. **Saturation phase (steps 200+)**: Growth slows as glucose depletion zones form around the tumor mass, spatial constraints limit expansion into occupied regions, and the terminal condition ($n_{\text{tumor}} \geq 2000$ or mean growth rate $\geq$ threshold) triggers simulation termination.

10.1.2 Expected Immune Response

The immune cell populations are expected to show a characteristic depletion pattern:

- **Cytotoxic T cells**: Initially active in killing tumor cells, but progressively exhausted through PD-1/PD-L1 checkpoint inhibition and the `w_progressive_exhaustion` parameter.
- **NK cells**: Provide early innate immune killing, but their effectiveness decreases as tumor cells establish PD-1 inhibition.
- **M1 macrophages**: Phagocytose tumor cells and present antigens, but may mutate to M2 phenotype under the influence of the tumor microenvironment.
- **M2 macrophages**: Accumulate over time as M1-to-M2 polarization occurs, promoting tumor growth and angiogenesis.
- **Tregs**: Differentiate from naïve CD4+ T cells and suppress anti-tumor immunity, further impairing immune control.

10.1.3 Glucose Field Behavior

The glucose field begins at a uniform concentration of 1.0 across all grid positions. As the simulation progresses, blood vessels inject glucose at their locations (rate `w_glucose_source_rate` = 5.0 per vessel per step), creating localized high-concentration zones. Tumor cells consume glucose at a high rate (`w_glucose_tumor_consumption` = 2.0 per cell per step), creating depletion zones around the growing tumor mass. The steady-state glucose profile is expected to show a characteristic spatial gradient: high near blood vessels, decreasing towards the tumor interior.

10.2 Treatment Scenarios

The model supports four treatment configurations, applied starting at step `treatment_start` (default 100). The treatment modalities are compared via their effects on tumor population dynamics and patient survival.

10.2.1 ICI Only

Immune checkpoint inhibitor therapy blocks the PD-1/PD-L1 pathway by setting `TumorCell.ICI_effec` = `True`, which causes `get_PD1_inhibition()` to return `False` unconditionally. The expected effects are:

- Partial restoration of cytotoxic T cell killing efficiency.
- Reduced T cell exhaustion, allowing sustained anti-tumor immune responses.
- Moderate tumor control: tumor growth rate decreases but may not be fully arrested, as ICI does not directly affect tumor cell proliferation or angiogenesis.

The model is expected to show a reduction in tumor cell count growth rate after step 100, with the immune cell populations stabilizing at higher levels compared to the baseline.

10.2.2 TKI Only

Tyrosine kinase inhibitor therapy targets the angiogenesis pathway by setting `TumorCell.TKI_effect = True`, which modulates tumor cell behavior upon apoptosis (releasing neoantigens to nearby dendritic cells). The expected effects are:

- Inhibition of new blood vessel formation, reducing glucose supply to the tumor.
- Indirect anti-tumor effects through glucose starvation, as the Warburg-dependent tumor cells are deprived of their metabolic substrate.
- Enhanced antigen presentation due to neoantigen release during TKI-induced apoptosis.

Simulation results are expected to show slower tumor growth compared to baseline, with the glucose depletion zone expanding more rapidly as blood vessel formation is inhibited.

10.2.3 ICI+TKI Combination

The combination therapy applies both ICI and TKI effects simultaneously. The expected synergistic benefits are:

- ICI restores immune checkpoint function while TKI reduces tumor blood supply.
- TKI-induced apoptosis releases neoantigens, which are more effectively recognized by the ICI-restored immune system.
- The glucose starvation effect of TKI compounds with the restored immune killing from ICI.

The combination is expected to produce the strongest tumor control, consistent with clinical evidence from the KEYNOTE-426 [12] and JAVELIN Renal 101 [10] trials that demonstrated superior outcomes for ICI+TKI combinations over monotherapy in advanced RCC.

10.2.4 Expected Visualizations

The simulation produces several types of output suitable for visualization:

1. **Cell count time series**: Population dynamics of all 18 agent types over simulation steps, plotted from the `simulation_log.csv` output. These plots show the rise and fall of tumor and immune cell populations under different treatment regimes.
2. **Survival curves**: Kaplan–Meier-style curves comparing survival outcomes across treatment groups, generated by running the model across multiple ARON-1 patient cases.
3. **Glucose field heatmaps**: 2D slices of the 3D glucose field at selected time points, showing the spatial distribution of glucose concentration and its evolution over time.

10.3 Sex-Specific Responses

The model captures sex-specific differences in immune response through three mechanisms: hormone environment, CD8+ T cell drift, and initial DNA mutation load.

10.3.1 Male vs. Female Comparison

- **Hormone environment**: Male patients have high testosterone (`chance = 1.0`) and low estrogen (`chance = 0.2`), while female patients have the inverse pattern (estrogen `chance = 0.9`, testosterone `chance = 0.2`). These hormones differentially affect CD4+ T cell differentiation, Treg activation, and macrophage polarization.

- **CD8+ drift**: Males undergo a positive drift in CD8+ naïve T cell population (drift sign $= +1$), while females undergo a negative drift (drift sign $= -1$). Over the first 15 steps, the drift adjusts the CD8+ population by up to `w_max_drift` $= 25\%$. This reflects epidemiological observations of sex differences in T cell counts [3].

- **Initial mutations**: Male patients' tumor cells are initialized with 3 injected mutations, while female patients start with 1. This models the observed higher mutational burden in male RCC patients, which can lead to both increased neoantigen presentation and more aggressive tumor behavior.

10.3.2 Expected Outcomes

The model is expected to show that male patients have a stronger initial CD8+ T cell response (due to positive drift and higher mutation-driven neoantigen presentation), but this advantage may be offset by the higher initial mutational burden promoting more aggressive tumor growth. Female patients may show a more balanced but sustained immune response due to estrogen-mediated immune modulation. The net effect on survival depends on the interplay of these factors, which is captured through the optimized weight parameters.

10.4 Glucose Field Dynamics

The glucose field provides a continuous metabolic overlay on the discrete agent grid, capturing the spatial competition for metabolic resources between tumor and immune cells.

10.4.1 Initial State

At step 0, the glucose field is initialized to a uniform concentration of 1.0 at every grid position. Blood vessels have been placed during initialization but have not yet injected glucose.

10.4.2 Source–Diffusion–Consumption Cycle

At each simulation step, the glucose field undergoes the following cycle:

1. **Source injection**: Each blood vessel agent injects `w_glucose_source_rate` = 5.0 units of glucose at its position. This creates localized "hot spots" of high glucose concentration near the vasculature.

2. **Diffusion**: The 3D Laplacian diffusion operator spreads glucose from high-concentration regions to low-concentration regions with coefficient $D = 0.1$. The zero-flux boundary conditions ensure that glucose does not leak out of the simulation domain.

3. **Decay**: A uniform decay of 1% per step ($d = 0.01$) is applied, modeling background tissue metabolism.

4. **Consumption**: During their individual `step()` calls, tumor cells consume `w_glucose_tumor_cons` = 2.0 per cell and immune cells consume `w_glucose_immune_consumption` = 0.5 per cell.

10.4.3 Spatial Profile Evolution

The glucose field is expected to develop a characteristic spatial profile over time:

- **Near blood vessels**: Glucose concentration remains high (well above 1.0 due to continuous injection), creating metabolically favorable zones.

- **Tumor interior**: Glucose is heavily depleted due to the Warburg effect, with concentrations approaching zero in densely populated tumor regions. This depletion reflects the clinical observation that solid tumors are metabolically hypoxic and glucose-starved in their cores [7].

- **Tumor periphery**: A gradient exists between the vessel-supplied exterior and the depleted interior. This gradient drives immune cell chemotaxis: with probability `w_glucose_chemotaxis_strength` = 0.3, immune cells move up the glucose gradient, which tends to direct them towards the tumor–vessel interface where metabolic resources are available.

- **Distant regions**: Glucose concentration equilibrates near the decay-adjusted injection rate, providing a metabolic "background" for immune cells that have not yet encountered the tumor.

10.4.4 Warburg Effect Manifestation

The elevated tumor consumption rate (4× that of immune cells) creates a metabolic asymmetry that has two consequences:

1. **Tumor growth suppression**: As the tumor mass grows, local glucose is depleted, reducing the glucose factor $\min(1, C \cdot \texttt{w_glucose_growth_sensitivity})$ and thereby suppressing further proliferation. This provides a natural negative feedback loop on tumor growth.

2. **Immune impairment**: Glucose depletion in the tumor microenvironment also reduces the effectiveness of infiltrating immune cells, which depend on glucose for activation and effector function. This creates a "metabolic immune checkpoint" that complements the receptor-based PD-1/PD-L1 checkpoint.

The data log records four glucose field statistics at each step: mean, total, minimum, and maximum concentration. These provide a time-series view of the global metabolic state of the simulation.

10.5 Validation

10.5.1 Comparison Against ARON-1 Data

The primary validation strategy compares the model's survival predictions against the ARON-1 clinical dataset (Section 9.5). For each patient case, the model is run with the patient's clinical features and the optimized weight parameters. The simulation's binary survival prediction and the number of steps to termination are compared against the clinical outcome.

10.5.2 Expected Accuracy

The Bayesian optimization procedure (Section 9.4) minimizes the mean squared error across all ARON-1 cases. After 250 Optuna trials, the model is expected to achieve:

- Correct survival prediction (binary match) for the majority of patient cases.
- Per-treatment-group accuracy that reflects the relative difficulty of predicting outcomes for each treatment modality.
- Convergence of the optimization error, visible in the trial error time series logged to `data/trial_errors.csv`.

10.5.3 Parameter Sensitivity Analysis

The Optuna optimization also provides implicit sensitivity information through the trial history. Parameters that are consistently selected near the bounds of their search range, or that show high variance across successful trials, are likely to have a strong influence on the model's behavior. In particular:

- **Tumor growth weights** (`w_tumor_growth_eff`, `w_tumor_growth_dna`): Expected to be among the most sensitive parameters, as they directly control the primary outcome variable.
- **Glucose parameters**: The balance between `w_glucose_source_rate` and `w_glucose_tumor_co` is expected to show high sensitivity, as it determines whether the glucose feedback loop effectively limits tumor growth.
- **Treatment effectiveness** (`w_ici_effectiveness`, `w_tki_effectiveness`): These parameters directly scale the treatment effect and are expected to be tightly constrained by the clinical data.
- **PD-1 inhibition weight** (`w_gene_pd1_inhibition`): Controls the strength of immune checkpoint evasion by tumor cells, with significant impact on the ICI treatment arm.

Formal sensitivity analysis (e.g., Sobol indices or Morris screening) can be performed as a post-processing step using the Optuna trial database, but is beyond the scope of the current implementation.

Chapter 11

Conclusion

This chapter summarizes the contributions of the RCC agent-based model with glucose sensing, discusses its current limitations, and outlines directions for future work.

11.1 Summary of Contributions

This project has produced the following contributions:

1. **Successful port from Mesa to Repast4Py.** The original Mesa-based RCC agent-based model [6] has been systematically ported to the Repast4Py framework [2]. This involved replacing Mesa's `MultiGrid` with Repast4Py's native 3D `SharedGrid`, implementing a custom spatial index (`defaultdict(list)`) with accompanying grid utility functions to compensate for the lack of built-in neighbor queries, converting all agents to the `repast4py.core.Agent(id, type, rank)` identity model, and adapting the scheduling system to Repast4Py's event-driven `schedule_runner`. The port preserves the full behavioral fidelity of the original model, including 18 agent types, the 16-gene DNA mutation system, PD-1/PD-L1 immune checkpoint interactions, and the effect propagation mechanism.

2. **Novel glucose microenvironment system.** A 3D continuous glucose concentration field has been designed and implemented as an overlay on the discrete agent grid. The system models glucose dynamics through three coupled processes: source injection from blood vessel agents, Fick's law diffusion via the discrete 3D Laplacian [5], and consumption by tumor cells (at elevated Warburg rates [14]) and immune cells. The implementation uses pure NumPy arrays with SciPy's `ndimage.convolve` for efficient vectorized computation, and enforces zero-flux (Neumann) boundary conditions consistent with the sealed tissue volume assumption.

3. **Integration of metabolic dynamics with immune–tumor interactions.** The glucose field is coupled bidirectionally with agent behavior. Tumor cell proliferation is modulated by local glucose availability through a multiplicative glucose factor, creating a natural negative feedback loop on growth. Immune cell effector function is similarly modulated by glucose concentration, capturing the concept of a "metabolic immune checkpoint" that complements the receptor-based PD-1/PD-L1 checkpoint. Immune cell chemotaxis toward glucose-rich regions provides a spatial coupling between metabolic gradients and cell migration patterns.

4. **Extended parameter space.** The original model's 84 weight parameters have been extended with 8 glucose-related parameters (diffusion coefficient, decay rate, source rate, tumor consumption, immune consumption, growth sensitivity, immune sensitivity, and chemotaxis strength), bringing the total to 92 tunable parameters. Each glucose parameter has been integrated into the Optuna search space with biologically motivated bounds (e.g., the diffusion coefficient is constrained below 1/6 for numerical stability).

5. **Maintained compatibility with the Optuna optimization pipeline.** The training infrastructure has been adapted to include the 8 new glucose parameters in the Bayesian optimization search space, while preserving compatibility with the ARON-1 clinical dataset and the existing evaluation methodology. The `suggest_parameters()` function, objective function, and trial logging have all been updated to accommodate the extended parameter space.

11.2 Model Limitations

Despite the contributions described above, the current implementation has several limitations that should be acknowledged:

1. **Single-rank execution.** Although Repast4Py supports multi-rank MPI parallelism, the current model operates on a single MPI rank (`mpirun -n 1`). The glucose field, in particular, would require halo exchange (ghost cell synchronization) to distribute the NumPy array across ranks, since the `scipy.ndimage.convolve` operation requires access to neighboring voxels that may reside on different ranks. Implementing this halo exchange is a non-trivial engineering task that has been deferred.

2. **Simplified glucose dynamics.** The glucose field models only a single metabolite (glucose) with linear diffusion and constant consumption rates. Real tumor microenvironments exhibit coupled dynamics involving oxygen (hypoxia), lactate (acidification from glycolysis), and amino acids. The current model does not capture these additional metabolites or the switch between aerobic and anaerobic glycolysis that characterizes the full Warburg phenotype.

3. **Fixed blood vessel topology.** While the model supports angiogenesis (creation of new blood vessels by tumor cells), the glucose source locations are determined by blood vessel agent positions at the time of injection. There is no dynamic remodeling of the vascular network in response to glucose field state or tumor growth pressure. Blood vessels, once placed, continue to inject glucose at a fixed rate regardless of surrounding conditions.

4. **No spatial heterogeneity in initial immune cell distribution.** All immune cells are initialized at random positions throughout the grid, without modeling the spatial organization of immune cell infiltration from the vascular periphery. In real tumors, immune cells enter the microenvironment through blood vessels and lymphatics, creating characteristic infiltration patterns (immune-excluded, immune-inflamed, immune-desert phenotypes) that are not captured.

5. **Simplified TCR matching model.** The T cell receptor matching system uses a discrete threshold-based comparison between antigen sequences and receptor sequences, with a configurable variation parameter. Real TCR–antigen interactions involve continuous binding affinities, cross-reactivity, and affinity maturation processes that are not modeled.

6. **Deferred multi-rank validation.** The `SharedContext` and `SharedGrid` infrastructure is in place for distributed execution, but the model has not been validated in multi-rank configurations. Ghost agent synchronization, spatial index consistency across rank boundaries, and glucose field halo exchange all require systematic testing before multi-rank deployment.

11.3 Future Work

Several directions for extending the model are identified:

1. **Multi-rank MPI with glucose field halo exchange.** The most impactful near-term extension would be to implement distributed glucose field computation using MPI halo exchange. Each rank would own a sub-volume of the glucose array, with ghost cells at rank boundaries synchronized after each diffusion step. This would enable simulation of larger tissue volumes (e.g., full tumor cross-sections at cellular resolution) that exceed the memory capacity of a single process.

2. **Additional metabolites.** Extending the glucose field to a multi-species reaction–diffusion system would capture oxygen–glucose coupling (hypoxia-driven metabolic switching), lactate production and its immunosuppressive effects, and amino acid competition (particularly arginine and tryptophan, which are depleted by tumor-associated enzymes such as arginase and IDO). Each additional metabolite would introduce a corresponding NumPy array with its own diffusion coefficient, source/sink terms, and agent coupling parameters.

3. **Dynamic blood vessel remodeling.** Future versions could model blood vessel pruning (regression of poorly perfused vessels), maturation (vessels becoming less permeable over time), and flow-dependent glucose delivery (vessels in high-flow regions deliver more glucose). This would create a dynamic feedback loop between the vascular network, glucose supply, and tumor growth.

4. **Spatial tumor heterogeneity and clonal evolution.** The DNA mutation system already supports heterogeneous mutations across tumor cells, but the current model does not explicitly model clonal evolution—the process by which genetically distinct subpopulations compete for space and resources within the tumor. Extending the model to track clonal lineages and their spatial distribution would enable study of therapy-resistant subclone emergence.

5. **Integration with real patient imaging data.** Initializing the model from patient-specific CT or MRI scans would allow simulation of realistic tumor geometries, blood vessel distributions, and organ boundaries. The 3D grid could be populated with voxel-level tissue classifications derived from imaging segmentation, providing a patient-specific digital twin for treatment planning.

6. **GPU acceleration for glucose field computation.** The glucose field's diffusion, decay, and consumption operations are embarrassingly parallel across the 3D grid. Porting these operations to GPU (e.g., via CuPy as a drop-in NumPy replacement, or JAX for automatic differentiation) would enable real-time simulation of large grids and could support gradient-based parameter optimization as an alternative or complement to Bayesian optimization.

7. **Extended validation with larger clinical cohorts.** The current calibration uses the ARON-1 dataset, which represents a specific patient population. Validating the model against additional clinical cohorts—including the CheckMate 214 [10], KEYNOTE-426 [12], and JAVELIN Renal 101 trials—would strengthen confidence in the model's generalizability. Cross-validation strategies (training on one cohort, testing on another) would provide rigorous assessment of overfitting.

8. **Formal sensitivity analysis.** Implementing Sobol indices or Morris screening on the 92-parameter space would quantify the relative importance of each parameter and identify parameter interactions. This analysis could guide parameter space reduction for faster optimization and identify the minimum set of patient features needed for clinically useful predictions.

In summary, the Repast4Py-based RCC agent-based model with glucose sensing represents a step toward more biologically realistic simulation of the tumor microenvironment. By integrating metabolic dynamics with the established immune–tumor interaction framework, the model captures an additional dimension of biological complexity that is increasingly recognized as central to cancer progression and treatment response [7, 14]. The modular architecture and compatibility with the Optuna optimization pipeline provide a foundation for systematic extension and clinical calibration.

Bibliography

[1] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama. Optuna: A next-generation hyperparameter optimization framework. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631, 2019.

[2] N. Collier, J. Ozik, and J.T. Murphy. Repast4py: A python framework for large-scale agent-based modeling. *Journal of Open Source Software*, 2023.

[3] F. Conforti et al. Cancer immunotherapy efficacy and patients' sex: A systematic review and meta-analysis. *The Lancet Oncology*, 19(6):737–746, 2018.

[4] L. Dalcin and Y.L. Fang. Mpi for python: Performance improvements and mpi modern features. *Journal of Parallel and Distributed Computing*, 171:34–90, 2023.

[5] A. Fick. *On Liquid Diffusion*. Philosophical Magazine, 1855.

[6] MASL Research Group. Agent-based learning model for renal cell carcinoma treatment optimization. 2024. Primary reference for the RCC-agent-learning-model implementation.

[7] D. Hanahan and R.A. Weinberg. Hallmarks of cancer: The next generation. *Cell*, 144(5):646–674, 2011.

[8] J. Kazil, D. Masad, and A. Crooks. Mesa: An agent-based modeling framework. *Journal of Open Source Software*, 5(55):2574, 2020.

[9] E. Merelli et al. Multi-level agent-based simulation for rcc immunotherapy modeling. 2021. Foundation paper for the multi-agent immune model.

[10] R.J. Motzer et al. Avelumab plus axitinib versus sunitinib for advanced renal-cell carcinoma. *New England Journal of Medicine*, 380(12):1103–1115, 2019.

[11] D.M. Pardoll. The blockade of immune checkpoints in cancer immunotherapy. *Nature Reviews Cancer*, 12(4):252–264, 2012.

[12] B.I. Rini et al. Pembrolizumab plus axitinib versus sunitinib for advanced renal-cell carcinoma. *New England Journal of Medicine*, 380(12):1116–1127, 2019.

[13] M. Santoni et al. Agent-based modeling of renal cell carcinoma tumor microenvironment: A computational approach to immune-tumor interactions. *Frontiers in Immunology*, 2024. MASL paper on RCC agent-based model with sex-specific immune responses.

[14] O. Warburg. On the origin of cancer cells. *Science*, 123(3191):309–314, 1956.